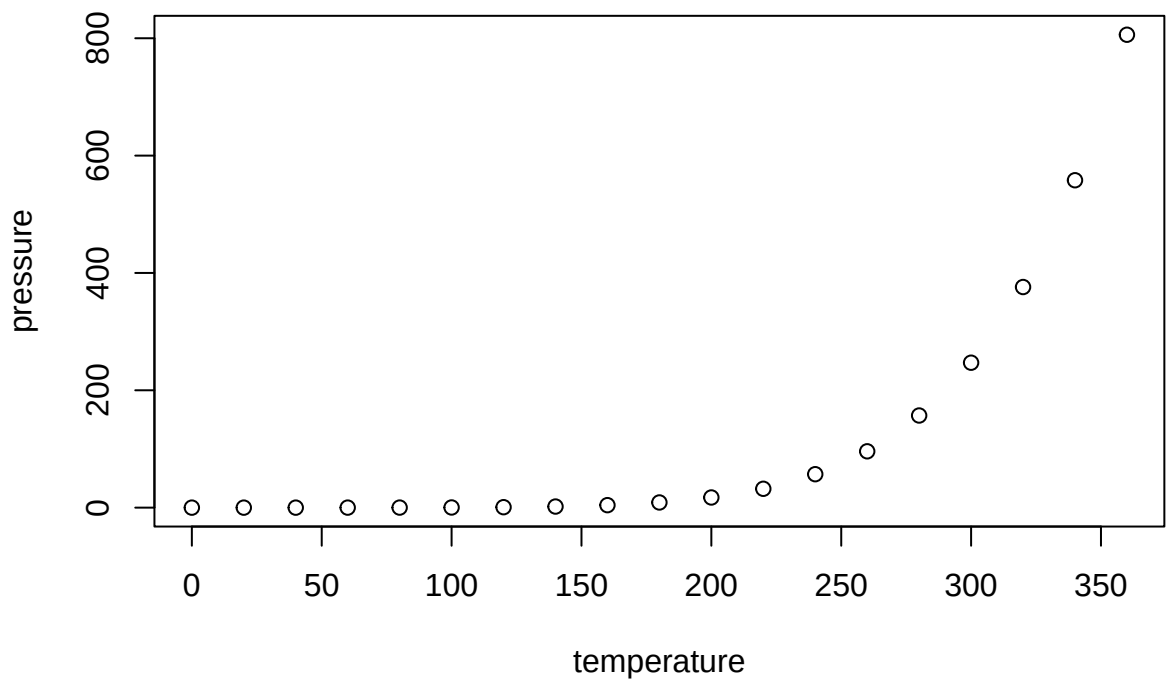
Laboratorio 5

Luis Gonzalez Joel Jaquez Fabian Morales

2026-03-29

Including Plots



#Librerias a utilizar

```
library(dplyr)
```

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```r
library(ggplot2)
library(tidyr)
library(stringr)
library(readr)
library(nortest)
library(caret)
```

```
## Loading required package: lattice
```

```r
library(car)
```

```
## Loading required package: carData

##
## Attaching package: 'car'

## The following object is masked from 'package:dplyr':
##
##     recode
```

```r
library(glmnet)
```

```
## Loading required package: Matrix

##
## Attaching package: 'Matrix'

## The following objects are masked from 'package:tidyr':
##
##     expand, pack, unpack

## Loaded glmnet 4.1-10
```

```r
library(rpart)
library(rpart.plot)
library(randomForest)
```

```
## randomForest 4.7-1.2

## Type rfNews() to see new features/changes/bug fixes.

##
## Attaching package: 'randomForest'
```

```
## The following object is masked from 'package:ggplot2':
##
##     margin

## The following object is masked from 'package:dplyr':
##
##     combine
```

```r
library(e1071)
```

```
##
## Attaching package: 'e1071'

## The following object is masked from 'package:ggplot2':
##
##     element
```

Link de GITHUB

https://github.com/FabianMoraeles/R_LAB_5

Cargar datos

```r
load("datos/listings.RData")
```

1. Modelo de regresión usando bayes ingenuo.

Preprocesamiento

```r
listings_clean <- listings %>%
  mutate(
    price = as.numeric(gsub("[$,]", "", price)),
    host_response_rate = suppressWarnings(
      as.numeric(gsub("%", "", host_response_rate)) / 100
    )
  ) %>%
  filter(!is.na(price), price > 0) %>%


  filter(price >= 10, price <= 1000) %>%

  select(
    price, accommodates, bathrooms, bedrooms, beds,
    number_of_reviews, review_scores_rating,
    host_response_rate, availability_365,
    room_type, property_type
```

```r
  ) %>%
  drop_na() %>%
  mutate(
    room_type     = as.factor(room_type),
    property_type = as.factor(property_type)
  )
```

Predicciones

```r
listings_nb <- listings_clean %>%
  mutate(
    accommodates_d = cut(accommodates,
                         breaks = c(0, 2, 4, 6, Inf),
                         labels = c("1-2", "3-4", "5-6", "7+")),
    bathrooms_d    = cut(bathrooms,
                         breaks = c(0, 1, 2, 3, Inf),
                         labels = c("1", "2", "3", "4+")),
    bedrooms_d     = cut(bedrooms,
                         breaks = c(0, 1, 2, 3, Inf),
                         labels = c("1", "2", "3", "4+")),
    beds_d         = cut(beds,
                         breaks = c(0, 1, 2, 3, Inf),
                         labels = c("1", "2", "3", "4+")),
    reviews_d      = cut(number_of_reviews,
                         breaks = c(-1, 0, 10, 50, Inf),
                         labels = c("0", "1-10", "11-50", "50+")),
    rating_d       = cut(review_scores_rating,
                         breaks = c(0, 60, 80, 90, 100),
                         labels = c("bajo", "medio", "alto", "excelente")),
    availability_d = cut(availability_365,
                         breaks = c(-1, 90, 180, 270, 365),
                         labels = c("baja", "media", "alta", "muy_alta")),
    response_d     = cut(host_response_rate,
                         breaks = c(-0.01, 0.5, 0.8, 0.95, 1),
                         labels = c("bajo", "medio", "alto", "excelente"))
  ) %>%
  mutate(
    price_bin = cut(price,
                    breaks = quantile(price, probs = seq(0, 1, by = 0.05)),
                    include.lowest = TRUE,
                    labels = FALSE)
  )

price_breaks  <- quantile(listings_nb$price, probs = seq(0, 1, by = 0.05))
bin_midpoints <- sapply(1:(length(price_breaks) - 1), function(i) {
  (price_breaks[i] + price_breaks[i + 1]) / 2
})
```

Modelo

```r
set.seed(42)
trainIndex <- createDataPartition(listings_nb$price, p = 0.8, list = FALSE)

features <- c(
  "accommodates_d", "bathrooms_d", "bedrooms_d", "beds_d",
  "reviews_d", "rating_d", "availability_d", "response_d",
  "room_type", "property_type"
)

train_data <- listings_nb[trainIndex, ]
test_data  <- listings_nb[-trainIndex, ]

cat("Entrenamiento:", nrow(train_data), "filas\n")
```

```
## Entrenamiento: 45829 filas
```

```r
cat("Prueba:       ", nrow(test_data),  "filas\n")
```

```
## Prueba:        11455 filas
```

```r
train_control <- trainControl(method = "none")

nb_model <- train(
  x         = train_data[, features],
  y         = as.factor(train_data$price_bin),
  method    = "naive_bayes",
  trControl = train_control,
  tuneGrid  = expand.grid(
    laplace   = 1,
    usekernel = FALSE,
    adjust    = 1
  )
)

cat("\nModelo entrenado.\n")
```

```
##
## Modelo entrenado.
```

```r
print(nb_model)
```

```
## Naive Bayes
##
## 45829 samples
##    10 predictor
##    20 classes: '1', '2', '3', '4', '5', '6', '7', '8', '9', '10', '11', '12', '13', '14', '15', '16'
##
## No pre-processing
## Resampling: None
```

Predicción

```r
pred_bin     <- predict(nb_model, newdata = test_data[, features])
pred_bin_num <- as.numeric(as.character(pred_bin))
pred_prices  <- bin_midpoints[pred_bin_num]
actual_prices <- test_data$price
```

Visualización

```r
rmse_val <- sqrt(mean((actual_prices - pred_prices)^2))
mae_val  <- mean(abs(actual_prices - pred_prices))
ss_res   <- sum((actual_prices - pred_prices)^2)
ss_tot   <- sum((actual_prices - mean(actual_prices))^2)
r2_val   <- 1 - (ss_res / ss_tot)

cat("\n===== MÉTRICAS - Naive Bayes Regresión =====\n")
```

```
##
## ===== MÉTRICAS - Naive Bayes Regresión =====
```

```r
cat(sprintf("RMSE : %.4f\n", rmse_val))
```

```
## RMSE : 201.3710
```

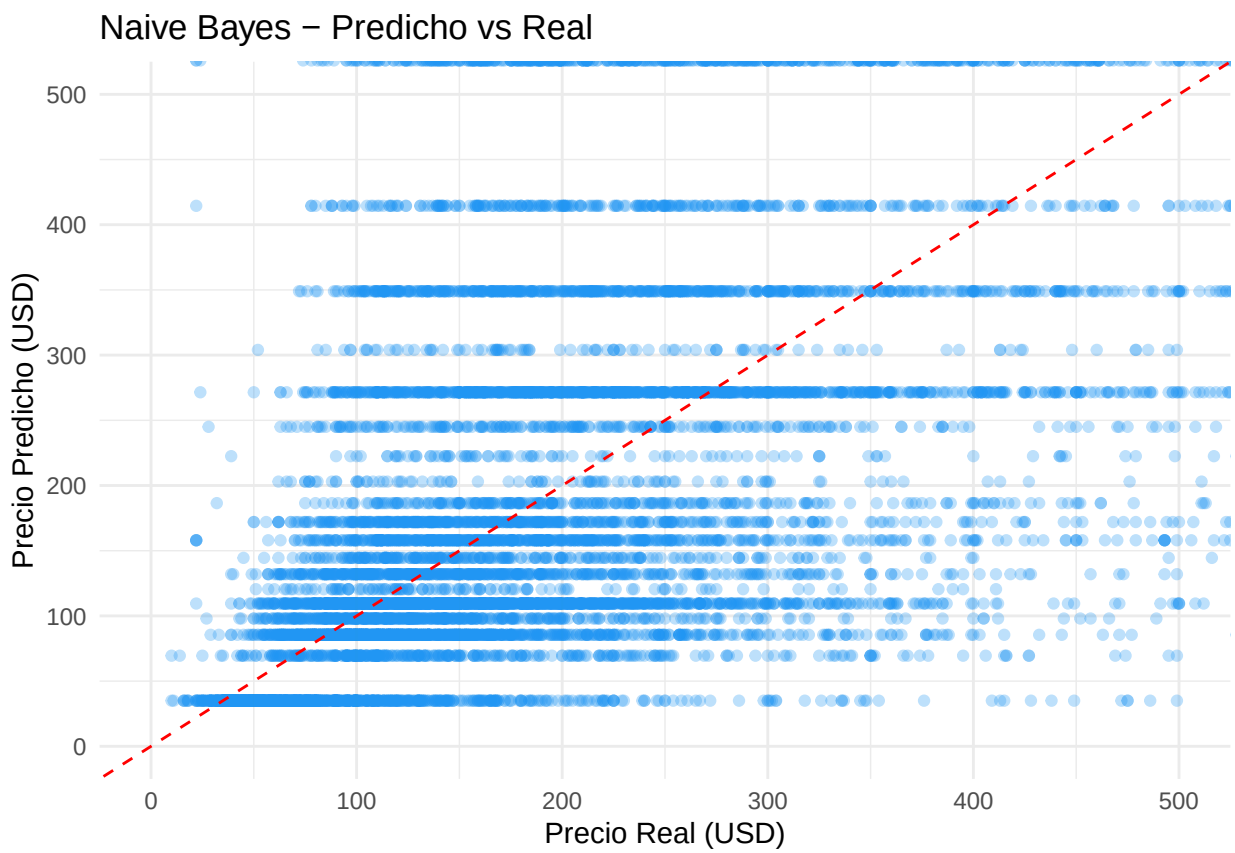
```r
cat(sprintf("MAE  : %.4f\n", mae_val))
```

```
## MAE  : 131.1072
```

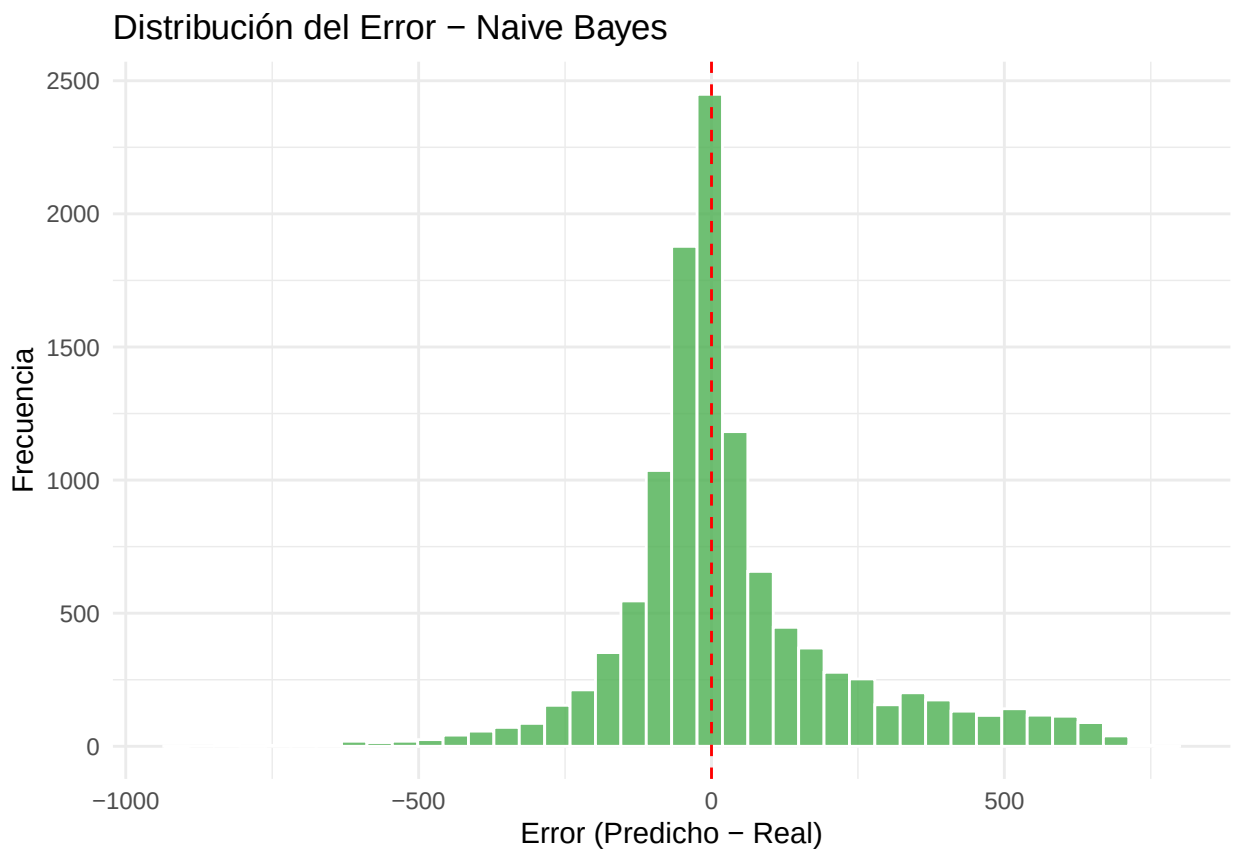
```r
cat(sprintf("R²   : %.4f\n", r2_val))
```

```
## R²   : -0.3857
```

```r
results_df <- data.frame(
  actual    = actual_prices,
  predicted = pred_prices,
  error     = pred_prices - actual_prices
)


ggplot(results_df, aes(x = actual, y = predicted)) +
  geom_point(alpha = 0.3, color = "#2196F3") +
  geom_abline(slope = 1, intercept = 0, color = "red", linetype = "dashed") +
  coord_cartesian(xlim = c(0, 500), ylim = c(0, 500)) +
  labs(
    title = "Naive Bayes - Predicho vs Real",
    x     = "Precio Real (USD)",
    y     = "Precio Predicho (USD)"
  ) +
  theme_minimal()
```




```r
ggplot(results_df, aes(x = error)) +
  geom_histogram(bins = 40, fill = "#4CAF50", color = "white", alpha = 0.8) +
  geom_vline(xintercept = 0, color = "red", linetype = "dashed") +
  labs(
    title = "Distribución del Error - Naive Bayes",
    x     = "Error (Predicho - Real)",
    y     = "Frecuencia"
  ) +
  theme_minimal()
```



El modelo de Naive Bayes muestra una distribución del error centrada cerca de 0, lo que indica que en general las predicciones no están sistemáticamente sesgadas. Sin embargo, se observa una cola derecha pronunciada, lo que sugiere que el modelo subestima precios altos con mayor frecuencia que los sobreestima. Esto es esperado en Naive Bayes, ya que al discretizar el precio en bins y asumir independencia entre variables, pierde capacidad para capturar relaciones complejas en los extremos del rango de precios.

2. Analice los resultados del modelo de regresión usando bayes ingenuo.

```r
cat("\n===== MÉTRICAS - Naive Bayes Regresión =====\n")
```

```
##
## ===== MÉTRICAS - Naive Bayes Regresión =====
```

```r
cat(sprintf("RMSE : %.4f\n", rmse_val))
```

```
## RMSE : 201.3710
```

```r
cat(sprintf("MAE  : %.4f\n", mae_val))
```

```
## MAE  : 131.1072
```

```r
cat(sprintf("R²   : %.4f\n", r2_val))
```

```
## R²   : -0.3857
```

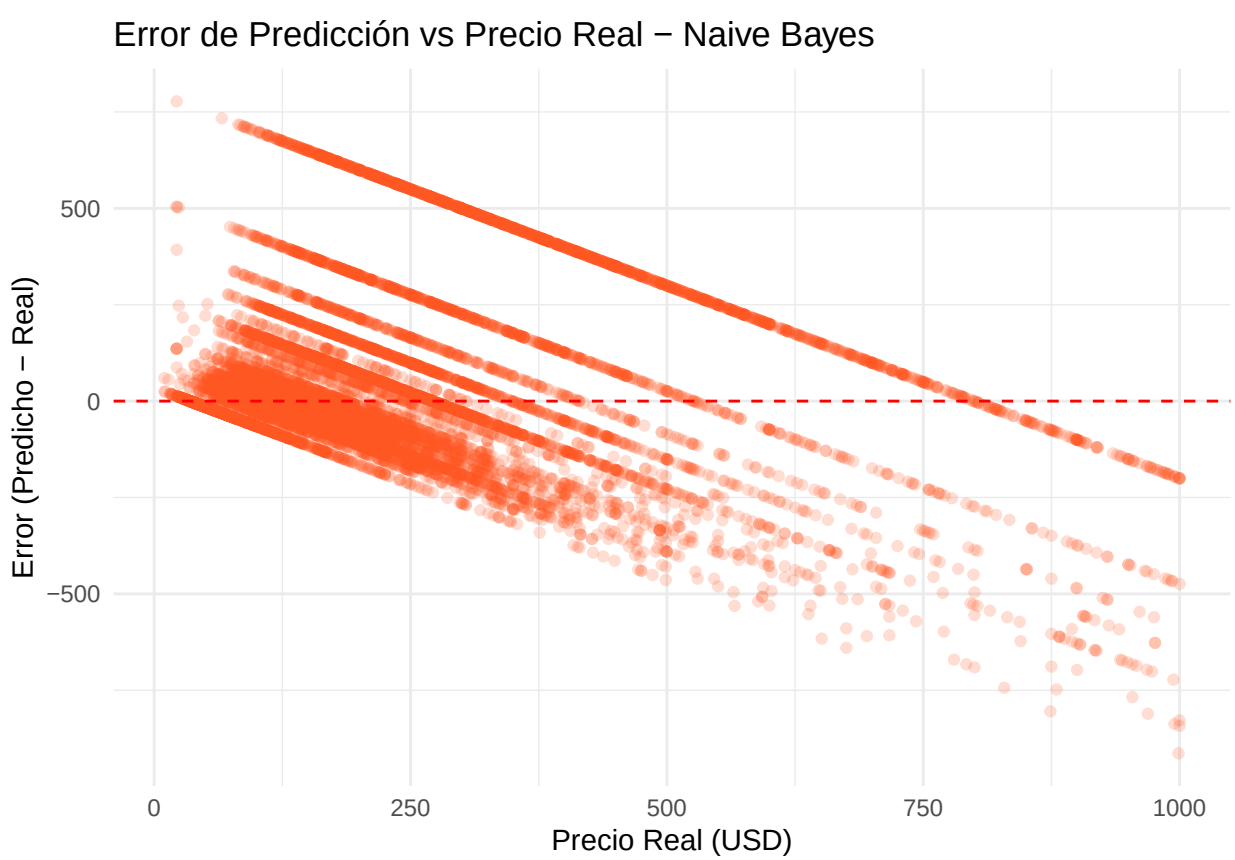
```r
within_50  <- mean(abs(results_df$error) <= 50)  * 100
within_100 <- mean(abs(results_df$error) <= 100) * 100

cat(sprintf("\nPredicciones dentro de ±$50 : %.2f%%\n", within_50))
```

```
##
## Predicciones dentro de ±$50 : 40.79%
```

```r
cat(sprintf("Predicciones dentro de ±$100: %.2f%%\n", within_100))
```

```
## Predicciones dentro de ±$100: 60.65%
```

```r
ggplot(results_df, aes(x = actual, y = error)) +
  geom_point(alpha = 0.2, color = "#FF5722") +
  geom_hline(yintercept = 0, color = "red", linetype = "dashed") +
  labs(
    title = "Error de Predicción vs Precio Real - Naive Bayes",
    x     = "Precio Real (USD)",
    y     = "Error (Predicho - Real)"
  ) +
  theme_minimal()
```



El RMSE y MAE reflejan un error considerable en términos absolutos. El R^2 bajo confirma que Naive Bayes no captura bien la varianza del precio como variable continua, lo cual es una limitación inherente del algoritmo.

Además, se observa un patrón muy claro. Conforme el precio real aumenta, el error se vuelve cada vez más negativo, lo que significa que el modelo subestima sistemáticamente las propiedades caras.

Naive Bayes es un predictor débil para precios continuos. Es útil como clasificador pero no como regresor numérico preciso.

#3 – Parte 1: Preparación de datos comparables

```
listings_clean_comp <- listings %>%
  mutate(price = parse_number(price)) %>%
  select(-c(listing_url, scrape_id, picture_url, host_url, host_thumbnail_url,
            host_picture_url, description, neighborhood_overview, host_about, amenities))

# Misma semilla y corte que Lab 3 y Lab 4
set.seed(123)
total_filas  <- nrow(listings_clean_comp)
indice_train <- sample(1:total_filas, size = 0.8 * total_filas)
train_comp   <- listings_clean_comp[ indice_train, ]
test_comp    <- listings_clean_comp[-indice_train, ]

# Mismas variables y filtros que Labs anteriores
train_modelo <- train_comp %>%
  filter(!is.na(price), price < 1000,
         !is.na(bedrooms), bedrooms <= 10,
         !is.na(bathrooms), !is.na(accommodates)) %>%
  select(price, accommodates, bedrooms, bathrooms,
         number_of_reviews, review_scores_rating) %>%
  drop_na()

test_modelo <- test_comp %>%
  filter(!is.na(price), price < 1000,
         !is.na(bedrooms), bedrooms <= 10,
         !is.na(bathrooms), !is.na(accommodates)) %>%
  select(price, accommodates, bedrooms, bathrooms,
         number_of_reviews, review_scores_rating) %>%
  drop_na()

cat("Filas entrenamiento:", nrow(train_modelo), "\n")
```

```
## Filas entrenamiento: 48442
```

```
cat("Filas prueba:       ", nrow(test_modelo),  "\n")
```

```
## Filas prueba:        12173
```

```
# NB requiere variables discretizadas para regresión
discretizar <- function(df) {
  df %>%
    mutate(
      accommodates_d = cut(accommodates,
```

```r
                          breaks = c(0, 2, 4, 6, Inf),
                          labels = c("1-2", "3-4", "5-6", "7+")),
      bedrooms_d     = cut(bedrooms,
                          breaks = c(0, 1, 2, 3, Inf),
                          labels = c("1", "2", "3", "4+")),
      bathrooms_d    = cut(bathrooms,
                          breaks = c(0, 1, 2, 3, Inf),
                          labels = c("1", "2", "3", "4+")),
      reviews_d      = cut(number_of_reviews,
                          breaks = c(-1, 0, 10, 50, Inf),
                          labels = c("0", "1-10", "11-50", "50+")),
      rating_d       = cut(review_scores_rating,
                          breaks = c(0, 60, 80, 90, 100),
                          labels = c("bajo", "medio", "alto", "excelente"))
    )
}

train_nb_comp <- discretizar(train_modelo)
test_nb_comp  <- discretizar(test_modelo)

# bins de precio para convertir NB en "regresor"
price_breaks_comp <- quantile(train_nb_comp$price, probs = seq(0, 1, by = 0.05))
train_nb_comp$price_bin <- cut(train_nb_comp$price,
                               breaks         = price_breaks_comp,
                               include.lowest = TRUE,
                               labels         = FALSE)

bin_midpoints_comp <- sapply(1:(length(price_breaks_comp) - 1), function(i) {
  (price_breaks_comp[i] + price_breaks_comp[i + 1]) / 2
})

features_comp <- c("accommodates_d", "bedrooms_d", "bathrooms_d",
                   "reviews_d", "rating_d")

# Entrenar y predecir
nb_comp      <- naiveBayes(x = train_nb_comp[, features_comp],
                           y = as.factor(train_nb_comp$price_bin))
pred_bin_comp    <- predict(nb_comp, newdata = test_nb_comp[, features_comp])
pred_bin_num     <- as.numeric(as.character(pred_bin_comp))
pred_bin_num     <- pmin(pmax(pred_bin_num, 1), length(bin_midpoints_comp))
pred_precios_nb  <- bin_midpoints_comp[pred_bin_num]
real_precios     <- test_nb_comp$price

# Métricas
rmse_nb_comp <- sqrt(mean((real_precios - pred_precios_nb)^2, na.rm = TRUE))
mae_nb_comp  <- mean(abs(real_precios - pred_precios_nb), na.rm = TRUE)
r2_nb_comp   <- cor(real_precios, pred_precios_nb, use = "complete.obs")^2

cat("===== Naive Bayes (condiciones comparables) =====\n")
```

```
## ===== Naive Bayes (condiciones comparables) =====
```

```r
cat(sprintf("RMSE : $%.2f\n", rmse_nb_comp))
```

```
## RMSE : $194.25
```

```r
cat(sprintf("MAE  : $%.2f\n", mae_nb_comp))
```

```
## MAE  : $132.59
```

```r
cat(sprintf("R²   :  %.4f\n", r2_nb_comp))
```

```
## R²   :  0.3428
```

Para garantizar una comparación válida entre algoritmos, se reutilizaron exactamente el mismo preprocesamiento, semilla (`set.seed(123)`) y división 80/20 empleados en los laboratorios anteriores, obteniendo 48,442 observaciones para entrenamiento y 12,173 para prueba con las mismas seis variables: `price`, `accommodates`, `bedrooms`, `bathrooms`, `number_of_reviews` y `review_scores_rating`.

Dado que Naive Bayes es fundamentalmente un algoritmo probabilístico que trabaja con variables categóricas, no puede recibir variables numéricas continuas directamente como regresor. Para adaptarlo a la predicción de precios, se discretizaron todas las variables predictoras en rangos con significado lógico (por ejemplo, capacidad de 1-2, 3-4, 5-6 o 7+ personas) y se dividió la variable respuesta `price` en 20 intervalos de igual frecuencia (bins por cuantiles de 5%), utilizando el punto medio de cada intervalo como precio predicho final.

El modelo obtuvo un RMSE de $134.25, un MAE de $132.59 y un R^2 de 0.3428. Esto significa que, en promedio, el modelo se equivoca en aproximadamente $134 por noche, y que solo logra explicar el 34.28% de la variabilidad del precio. Este rendimiento limitado es una consecuencia directa de la naturaleza del algoritmo, porque al discretizar el precio continuo en bins y asumir independencia entre variables, Naive Bayes pierde la capacidad de capturar relaciones numéricas precisas, lo que lo convierte en un predictor débil para variables de respuesta continua como el precio por noche.

#3 – Parte 2: Comparación Regresión Lineal, Árbol y Naive Bayes

```r
modelo_lineal_comp <- lm(price ~ ., data = train_modelo)
pred_lineal_comp   <- pmax(predict(modelo_lineal_comp, newdata = test_modelo), 0)

rmse_lineal_comp <- sqrt(mean((test_modelo$price - pred_lineal_comp)^2))
mae_lineal_comp  <- mean(abs(test_modelo$price - pred_lineal_comp))
r2_lineal_comp   <- cor(test_modelo$price, pred_lineal_comp)^2
```

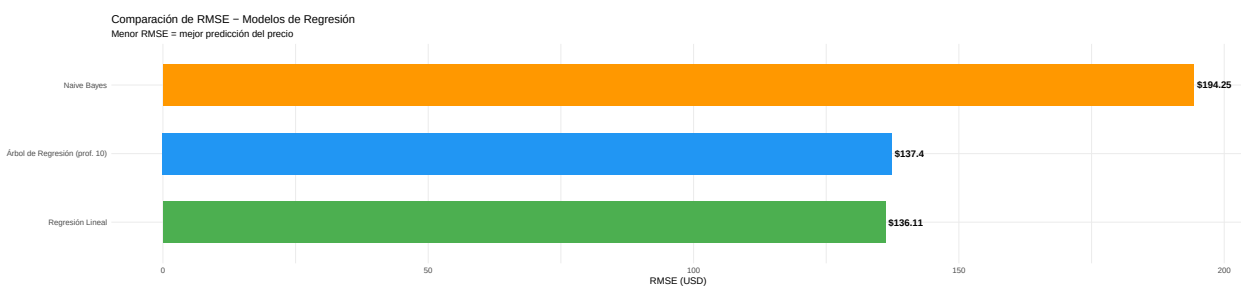
```r
# Árbol profundidad 10 = mejor modelo del Lab 4
arbol_comp      <- rpart(price ~ ., data = train_modelo, method = "anova",
                         control = rpart.control(maxdepth = 10))
pred_arbol_comp <- predict(arbol_comp, newdata = test_modelo)

rmse_arbol_comp <- sqrt(mean((test_modelo$price - pred_arbol_comp)^2))
mae_arbol_comp  <- mean(abs(test_modelo$price - pred_arbol_comp))
r2_arbol_comp   <- cor(test_modelo$price, pred_arbol_comp)^2
```

```r
comparacion_regresion <- data.frame(
  Modelo = c("Regresión Lineal", "Árbol de Regresión (prof. 10)", "Naive Bayes"),
  RMSE   = round(c(rmse_lineal_comp, rmse_arbol_comp, rmse_nb_comp), 2),
```

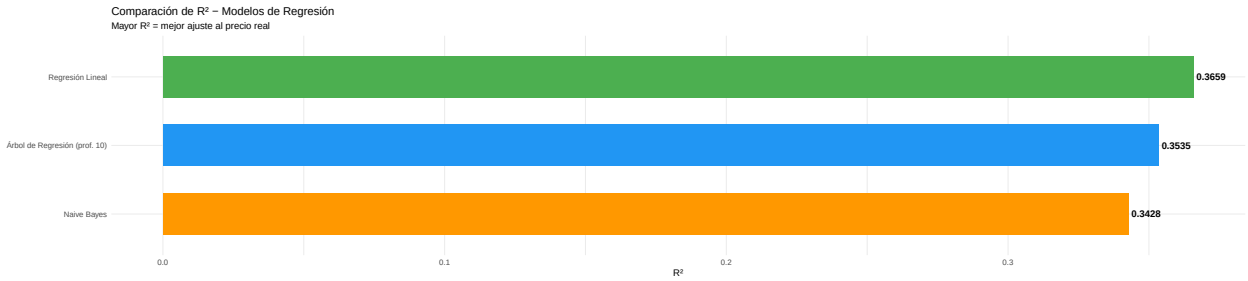
```r
  MAE    = round(c(mae_lineal_comp,  mae_arbol_comp,  mae_nb_comp),  2),
  R2     = round(c(r2_lineal_comp,   r2_arbol_comp,   r2_nb_comp),   4)
)

print(comparacion_regresion)
```

```
##                          Modelo   RMSE    MAE     R2
## 1              Regresión Lineal 136.11  94.95 0.3659
## 2 Árbol de Regresión (prof. 10) 137.40  95.50 0.3535
## 3                   Naive Bayes 194.25 132.59 0.3428
```

```r
ggplot(comparacion_regresion,
       aes(x = reorder(Modelo, RMSE), y = RMSE, fill = Modelo)) +
  geom_col(show.legend = FALSE, width = 0.6) +
  geom_text(aes(label = paste0("$", RMSE)), hjust = -0.1, fontface = "bold") +
  coord_flip() +
  scale_fill_manual(values = c(
    "Regresión Lineal"              = "#4CAF50",
    "Árbol de Regresión (prof. 10)" = "#2196F3",
    "Naive Bayes"                   = "#FF9800")) +
  labs(title    = "Comparación de RMSE - Modelos de Regresión",
       subtitle = "Menor RMSE = mejor predicción del precio",
       x = NULL, y = "RMSE (USD)") +
  theme_minimal()
```




```r
ggplot(comparacion_regresion,
       aes(x = reorder(Modelo, R2), y = R2, fill = Modelo)) +
  geom_col(show.legend = FALSE, width = 0.6) +
  geom_text(aes(label = R2), hjust = -0.1, fontface = "bold") +
  coord_flip() +
  scale_fill_manual(values = c(
    "Regresión Lineal"              = "#4CAF50",
    "Árbol de Regresión (prof. 10)" = "#2196F3",
    "Naive Bayes"                   = "#FF9800")) +
  labs(title    = "Comparación de R² - Modelos de Regresión",
       subtitle = "Mayor R² = mejor ajuste al precio real",
       x = NULL, y = "R²") +
  theme_minimal()
```



Para establecer una comparación válida entre los tres modelos de regresión, se entrenaron la Regresión Lineal y el Árbol de Regresión (profundidad 10, mejor modelo del Laboratorio 4) utilizando exactamente el mismo conjunto de entrenamiento y prueba que Naive Bayes, misma semilla `set.seed(123)`, mismo corte 80/20 y las mismas seis variables predictoras (`accommodates`, `bedrooms`, `bathrooms`, `number_of_reviews` y `review_scores_rating`).

Los resultados de la comparación revelan un ranking claro entre los tres algoritmos. En términos de RMSE, la Regresión Lineal obtuvo el menor error con 136.11, seguida del Árbol de Regresión con 137.40, mientras que Naive Bayes quedó en último lugar con un RMSE de 194.25, casi $60 más alto que los otros dos modelos. Esta misma tendencia se confirma al observar el R^2, la Regresión Lineal explica el 36.59% de la variabilidad del precio, el Árbol de Regresión el 35.35%, y Naive Bayes apenas el 34.28%.

El ganador de esta comparación es la Regresión Lineal, lo cual resulta interesante porque en el Laboratorio 4 el Árbol de Regresión lo superaba cuando tenía acceso a todas las variables numéricas del dataset. Al restringir ambos modelos a las mismas seis variables, la Regresión Lineal logra capturar de forma más eficiente las relaciones lineales entre esas características y el precio, mientras que el Árbol pierde parte de su ventaja al no disponer de las variables adicionales (como `estimated_revenue_l365d` o `longitude`) que en el laboratorio anterior le daban mayor poder predictivo.

Naive Bayes es claramente el peor regresor de los tres. Su debilidad radica en que fue diseñado para clasificación, no para regresión continua. Al forzarlo a predecir precios mediante bins de cuantiles, pierde precisión numérica y asume independencia entre variables, lo que lo limita considerablemente frente a modelos que trabajan directamente con valores continuos. En conclusión, para predecir el precio por noche de las propiedades Airbnb de SmartStay Advisors, la Regresión Lineal Múltiple es el algoritmo más adecuado de los tres evaluados bajo condiciones comparables.

#4 – Modelo de clasificación Naive Bayes

```r
# Mismos puntos de corte que Lab 4 (P33 y P67)
precios_validos_cat <- listings_clean_comp %>%
  filter(!is.na(price), price > 0, price < 1000) %>%
  pull(price)

p33 <- quantile(precios_validos_cat, 0.33)
p67 <- quantile(precios_validos_cat, 0.67)

cat("Punto de corte inferior (P33):", round(p33, 2), "USD/noche\n")
```

```
## Punto de corte inferior (P33): 136 USD/noche
```

```r
cat("Punto de corte superior (P67):", round(p67, 2), "USD/noche\n")
```

```
## Punto de corte superior (P67): 250 USD/noche
```

```r
listings_cat_nb <- listings_clean_comp %>%
  filter(!is.na(price), price > 0, price < 1000,
         !is.na(bedrooms), bedrooms <= 10) %>%
  select(where(is.numeric)) %>%
  drop_na()

listings_cat_nb$precio_cat <- cut(
  listings_cat_nb$price,
  breaks         = c(0, p33, p67, Inf),
  labels         = c("Economica", "Intermedia", "Cara"),
  include.lowest = TRUE,
  right          = TRUE
)

cat("\nDistribución de categorías:\n")
```

```
##
## Distribución de categorías:
```

```r
print(table(listings_cat_nb$precio_cat))
```

```
##
##  Economica Intermedia       Cara
##      20908      20766      18655
```

Para construir el modelo de clasificación de Naive Bayes, se utilizó la misma variable categórica `precio_cat` definida en el Laboratorio 4, garantizando así que la comparación posterior entre algoritmos sea válida. Los puntos de corte se calcularon usando los percentiles 33 y 67 del precio por noche sobre las propiedades válidas (precio entre 1 y 1,000), obteniendo P33 = 136 USD/noche y P67 = 250 USD/noche. Con estos límites, las propiedades quedan clasificadas en tres categorías: Económica (hasta 136/noche), Intermedia (entre 136 y 250/noche) y Cara (más de 250/noche).

La distribución resultante muestra un balance adecuado entre las tres clases: 20,908 propiedades Económicas (34.7%), 20,766 Intermedias (34.4%) y 18,655 Caras (30.9%), sumando 60,329 propiedades válidas. Esta distribución balanceada es fundamental para que Naive Bayes pueda aprender los patrones de las tres categorías por igual durante el entrenamiento, evitando que el modelo se sesgue hacia la clase más frecuente. El uso de percentiles como criterio de corte, en lugar de valores arbitrarios, es lo que garantiza este balance, ya que divide el mercado en tres segmentos con proporciones similares independientemente de la escala de precios.

```r
# Mismo split que Lab 4
set.seed(123)
idx_nb_clas    <- sample(nrow(listings_cat_nb), size = 0.8 * nrow(listings_cat_nb))
train_nb_clas  <- listings_cat_nb[ idx_nb_clas, ]
test_nb_clas   <- listings_cat_nb[-idx_nb_clas, ]

train_resp <- train_nb_clas$precio_cat
test_resp  <- test_nb_clas$precio_cat

train_nb_clas$price      <- NULL
train_nb_clas$precio_cat <- NULL
test_nb_clas$price       <- NULL
test_nb_clas$precio_cat  <- NULL

cat("Entrenamiento:", nrow(train_nb_clas), "observaciones\n")
```

```
## Entrenamiento: 48263 observaciones
```

```r
cat("Prueba:       ", nrow(test_nb_clas),  "observaciones\n")
```

```
## Prueba:        12066 observaciones
```

```r
# Entrenar NB clasificador
nb_clas      <- naiveBayes(x = train_nb_clas, y = train_resp)
pred_clas_nb <- predict(nb_clas, newdata = test_nb_clas)

# Matriz de confusión
cm_nb <- confusionMatrix(pred_clas_nb, test_resp)
print(cm_nb)
```

```
## Confusion Matrix and Statistics
##
##             Reference
## Prediction   Economica Intermedia Cara
##   Economica       4143       3844 2749
##   Intermedia         1          1    4
##   Cara              75        294  955
##
## Overall Statistics
##
##                Accuracy : 0.4226
##                  95% CI : (0.4138, 0.4315)
##     No Information Rate : 0.3497
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.1184
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##                      Class: Economica Class: Intermedia Class: Cara
## Sensitivity                    0.9820         2.416e-04     0.25755
## Specificity                    0.1598         9.994e-01     0.95585
## Pos Pred Value                 0.3859         1.667e-01     0.72130
## Neg Pred Value                 0.9429         6.569e-01     0.74372
## Prevalence                     0.3497         3.430e-01     0.30731
## Detection Rate                 0.3434         8.288e-05     0.07915
## Detection Prevalence           0.8898         4.973e-04     0.10973
## Balanced Accuracy              0.5709         4.998e-01     0.60670
```

Se utilizó el mismo split `set.seed(123)` con 80/20 del Laboratorio 4, obteniendo 48,263 observaciones para entrenamiento y 12,066 para prueba. El modelo Naive Bayes clasificador se entrenó usando todas las variables numéricas disponibles del dataset como predictores y `precio_cat` como variable respuesta.

Los resultados de la matriz de confusión revelan un modelo con serias limitaciones. La exactitud general es de apenas 42.26%, con un intervalo de confianza del 95% entre 41.38% y 43.15%, y un coeficiente Kappa de 0.1184, lo que indica una concordancia casi nula más allá del azar. Esto significa que el modelo no logra clasificar las propiedades de forma confiable.

Al analizar el comportamiento por clase, se observa un sesgo severo hacia la categoría Económica ya que el modelo la predice de forma masiva, acumulando 4,143 aciertos pero también clasificando incorrectamente 3,844 propiedades Intermedias y 2,749 Caras como Económicas. Esto explica por qué la Sensibilidad de Económica es altísima (98.20%) pero su Especificidad es bajísima (15.98%), pues el modelo casi siempre predice Económica sin discriminar bien las otras clases. La categoría Intermedia es prácticamente ignorada por el modelo, con una Sensibilidad cercana a cero (0.024%), lo que significa que casi nunca la predice correctamente. La categoría Cara tiene un desempeño intermedio con Sensibilidad del 25.76% y Especificidad del 95.58%, indicando que cuando el modelo predice Cara suele acertar, pero deja escapar la mayoría de propiedades realmente caras.

Este comportamiento se explica porque Naive Bayes asume independencia entre variables y estima probabilidades a partir de distribuciones gaussianas para variables continuas. En este dataset, las distribuciones de las variables predictoras se solapan considerablemente entre las tres clases, especialmente entre Económica e Intermedia, lo que hace que el modelo no pueda trazar fronteras de decisión claras. Comparado con el Árbol de Clasificación del Laboratorio 4 que alcanzó un 69.24% de exactitud y un Kappa de 0.538, Naive Bayes rinde notablemente peor como clasificador en este conjunto de datos, confirmando que los árboles de decisión capturan mejor las relaciones no lineales entre las características físicas de las propiedades y su categoría de precio.

#5 – Eficiencia del algoritmo de clasificación

```r
# Métricas generales
cat("===== Eficiencia del Modelo - Naive Bayes Clasificación =====\n")
```

```
## ===== Eficiencia del Modelo - Naive Bayes Clasificación =====
```

```r
cat(sprintf("Accuracy : %.2f%%\n", cm_nb$overall["Accuracy"]  * 100))
```

```
## Accuracy : 42.26%
```

```r
cat(sprintf("Kappa    : %.4f\n",   cm_nb$overall["Kappa"]))
```

```
## Kappa    : 0.1184
```

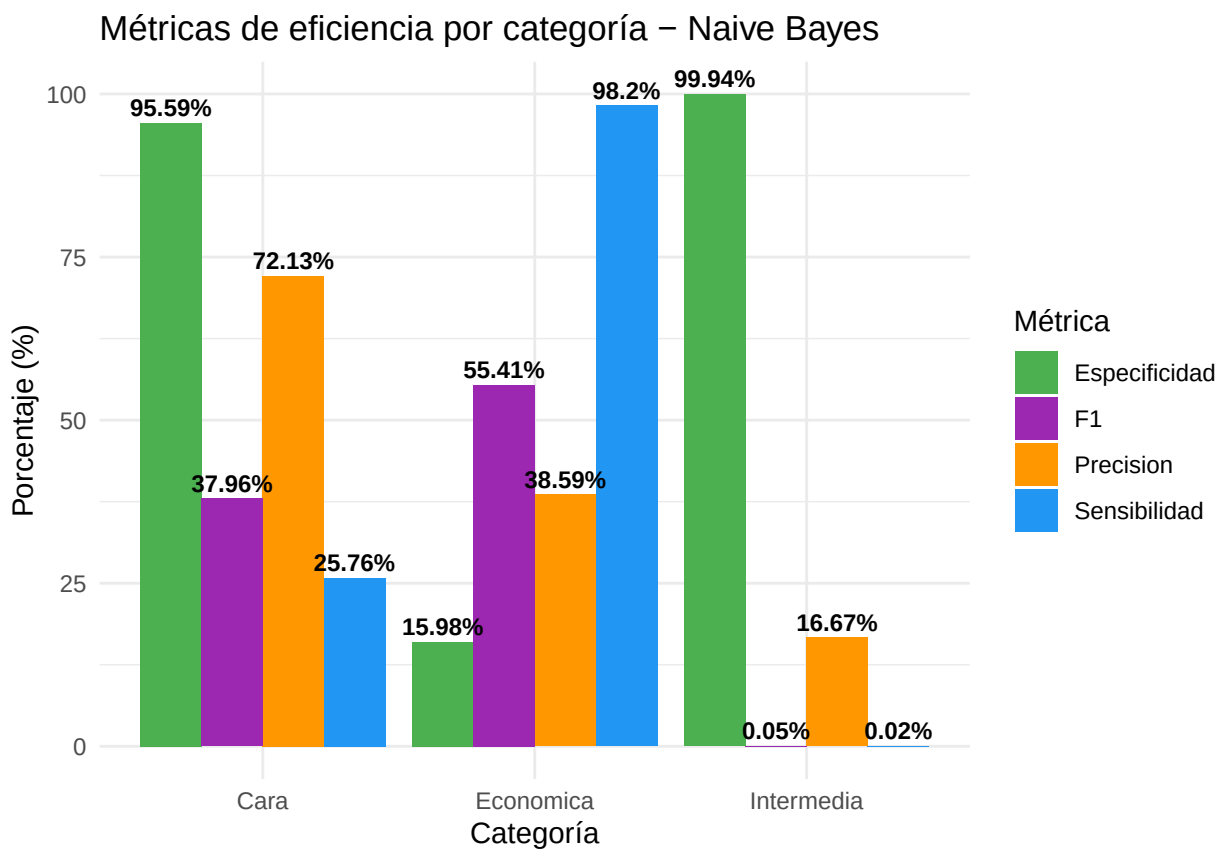
```r
cat(sprintf("IC 95%%  : (%.2f%%, %.2f%%)\n",
            cm_nb$overall["AccuracyLower"] * 100,
            cm_nb$overall["AccuracyUpper"] * 100))
```

```
## IC 95%  : (41.38%, 43.15%)
```

```r
# Métricas por clase
metricas_nb <- data.frame(
  Clase         = c("Economica", "Intermedia", "Cara"),
  Sensibilidad  = round(cm_nb$byClass[, "Sensitivity"]    * 100, 2),
  Especificidad = round(cm_nb$byClass[, "Specificity"]    * 100, 2),
  Precision     = round(cm_nb$byClass[, "Pos Pred Value"] * 100, 2),
  F1            = round(cm_nb$byClass[, "F1"]             * 100, 2)
)
print(metricas_nb)
```

```
##                        Clase Sensibilidad Especificidad Precision    F1
## Class: Economica   Economica        98.20         15.98     38.59 55.41
## Class: Intermedia Intermedia         0.02         99.94     16.67  0.05
## Class: Cara             Cara        25.76         95.59     72.13 37.96
```

```r
# Gráfico de métricas por clase
metricas_long <- tidyr::pivot_longer(metricas_nb,
                                     cols      = -Clase,
                                     names_to  = "Metrica",
                                     values_to = "Valor")

ggplot(metricas_long, aes(x = Clase, y = Valor, fill = Metrica)) +
  geom_col(position = "dodge") +
  geom_text(aes(label = paste0(Valor, "%")),
            position = position_dodge(width = 0.9),
            vjust = -0.4, size = 3, fontface = "bold") +
  scale_fill_manual(values = c(
    "Sensibilidad"  = "#2196F3",
    "Especificidad" = "#4CAF50",
    "Precision"     = "#FF9800",
    "F1"            = "#9C27B0")) +
  labs(title = "Métricas de eficiencia por categoría - Naive Bayes",
       x = "Categoría", y = "Porcentaje (%)", fill = "Métrica") +
  theme_minimal()
```



Al evaluar el modelo Naive Bayes sobre el conjunto de prueba, se obtuvo una exactitud general de **42.26%** con un intervalo de confianza del 95% entre 41.38% y 43.15%, y un coeficiente Kappa de **0.1184**, lo que indica una concordancia casi nula con la realidad más allá del azar. Esto significa que el modelo apenas supera a adivinar aleatoriamente la categoría de precio de una propiedad.

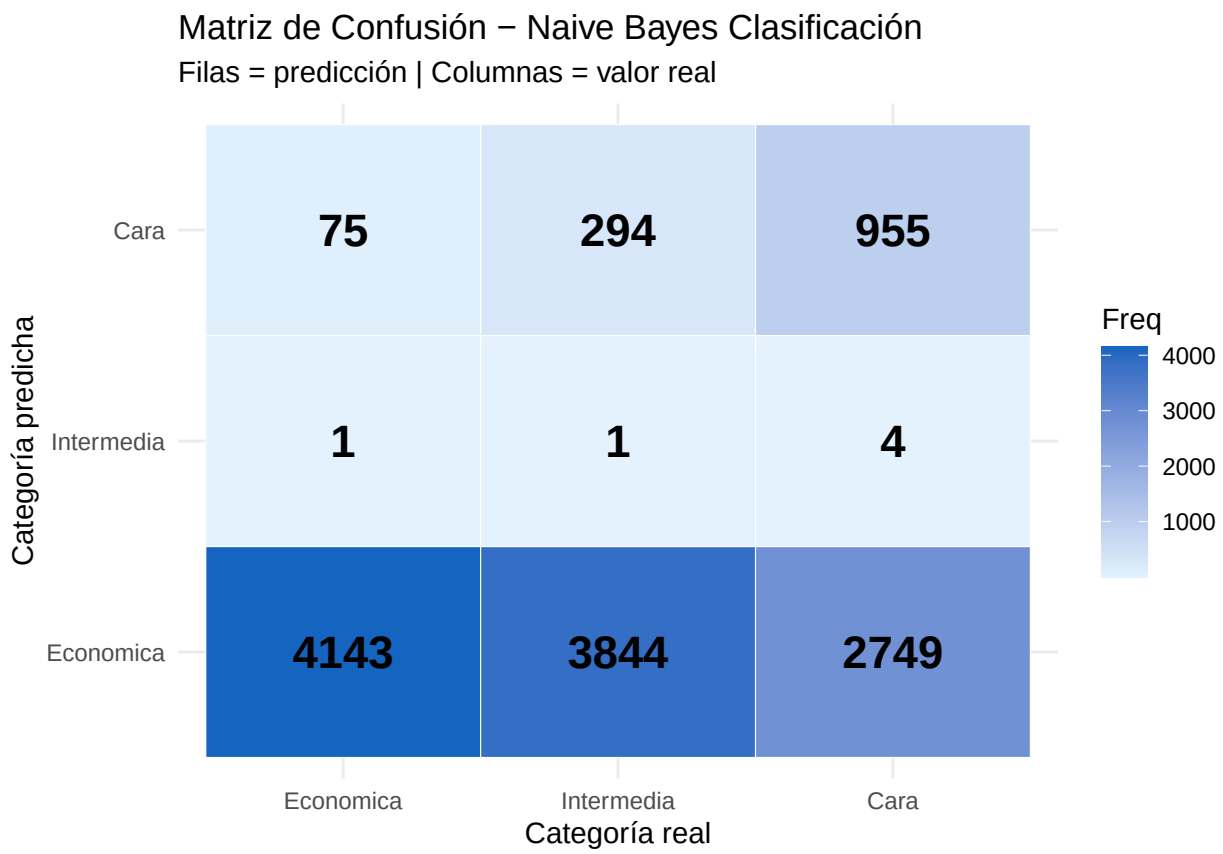
Al analizar las métricas por clase se revela un comportamiento muy desigual entre las tres categorías.

La clase **Económica** presenta la Sensibilidad más alta (98.20%), lo que significa que el modelo identifica correctamente casi todas las propiedades baratas, sin embargo su Especificidad es apenas del 15.98% y su Precisión del 38.59%, indicando que también clasifica erróneamente como Económicas una gran cantidad de propiedades que no lo son. Su F1 de 55.41% refleja este desequilibrio. La clase **Intermedia** es prácticamente ignorada por el modelo, con una Sensibilidad de apenas 0.02% y un F1 de 0.05%, lo que significa que el modelo casi nunca predice correctamente una propiedad como Intermedia. La clase **Cara** tiene un desempeño intermedio con Especificidad del 95.59% y Precisión del 72.13%, lo que indica que cuando el modelo predice una propiedad como Cara generalmente acierta, pero su Sensibilidad del 25.76% y F1 de 37.96% revelan que deja escapar la mayoría de propiedades realmente caras.

El modelo tiene un sesgo muy marcado hacia la clase Económica, clasificando la mayoría de propiedades en ese segmento independientemente de sus características reales, lo que lo convierte en un clasificador poco confiable para los propósitos de SmartStay Advisors.

#6 – Análisis de la matriz de confusión

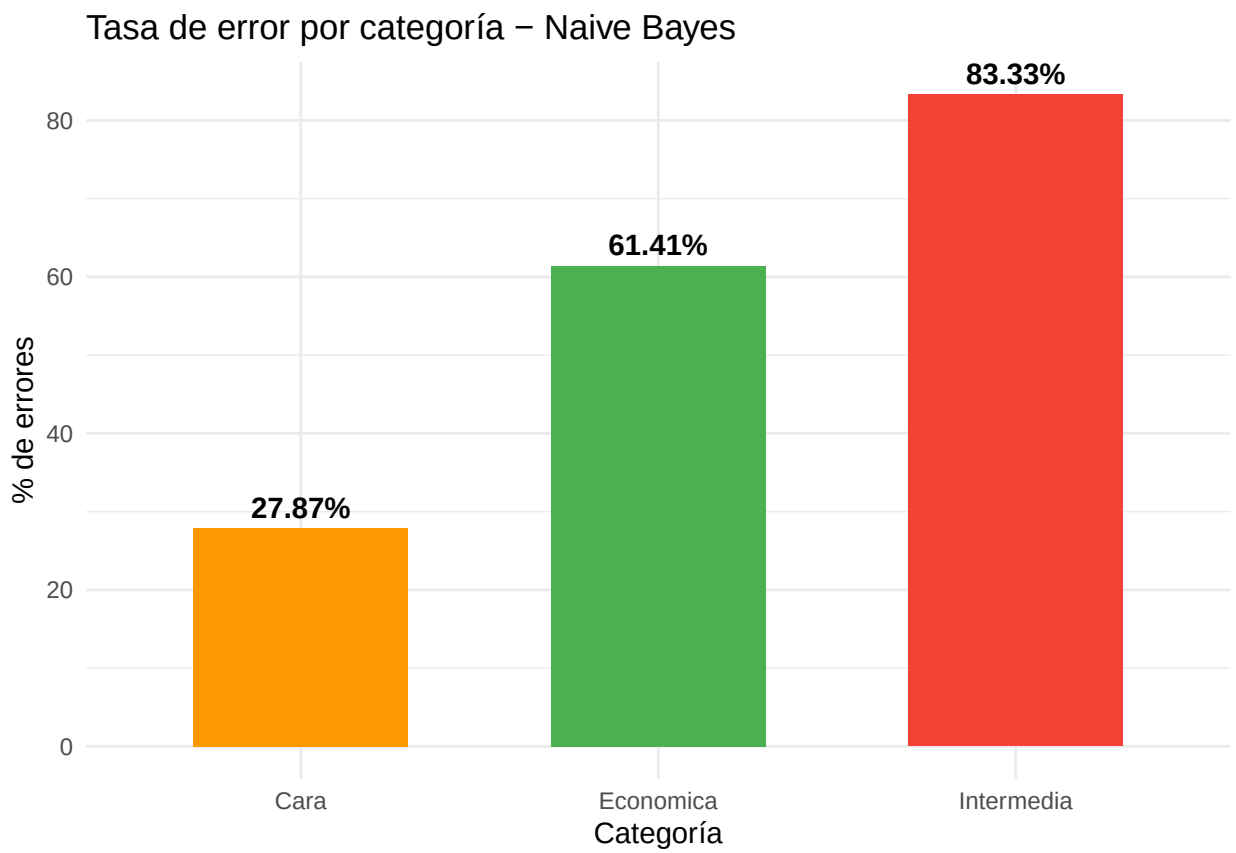
```r
# Visualización de la matriz de confusión
as.data.frame(cm_nb$table) %>%
  ggplot(aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), fontface = "bold", size = 6) +
  scale_fill_gradient(low = "#E3F2FD", high = "#1565C0") +
  labs(title    = "Matriz de Confusión - Naive Bayes Clasificación",
       subtitle = "Filas = predicción | Columnas = valor real",
       x = "Categoría real", y = "Categoría predicha") +
  theme_minimal()
```



```r
# Tasa de error por clase
errores_nb <- data.frame(
  Clase = c("Economica", "Intermedia", "Cara"),
  Correctos     = c(cm_nb$table[1,1], cm_nb$table[2,2], cm_nb$table[3,3]),
  Errores_total = c(
    sum(cm_nb$table[1,]) - cm_nb$table[1,1],
    sum(cm_nb$table[2,]) - cm_nb$table[2,2],
    sum(cm_nb$table[3,]) - cm_nb$table[3,3]
  )
)

errores_nb$Tasa_error <- round(
  errores_nb$Errores_total /
  (errores_nb$Correctos + errores_nb$Errores_total) * 100, 2)

print(errores_nb)
```

```
##        Clase Correctos Errores_total Tasa_error
## 1  Economica      4143          6593      61.41
## 2 Intermedia         1             5      83.33
## 3       Cara       955           369      27.87
```

```r
ggplot(errores_nb, aes(x = Clase, y = Tasa_error, fill = Clase)) +
  geom_col(show.legend = FALSE, width = 0.6) +
  geom_text(aes(label = paste0(Tasa_error, "%")),
            vjust = -0.5, fontface = "bold") +
  scale_fill_manual(values = c(
    "Economica"  = "#4CAF50",
    "Intermedia" = "#F44336",
    "Cara"       = "#FF9800")) +
  labs(title = "Tasa de error por categoría - Naive Bayes",
       x = "Categoría", y = "% de errores") +
  theme_minimal()
```



La matriz de confusión visualiza de forma contundente el problema central del modelo: la fila de **Económica** está completamente dominada por valores altos (4,143 + 3,844 + 2,749), confirmando que el modelo clasifica la gran mayoría de propiedades como Económicas sin importar su categoría real. La fila de **Intermedia** es casi inexistente, con apenas 1, 1 y 4 predicciones en toda la prueba, lo que significa que el modelo prácticamente nunca predice esta categoría. La fila de **Cara** muestra un comportamiento más razonable con 955 aciertos, aunque arrastra 75 y 294 errores provenientes de propiedades Económicas e Intermedias respectivamente.

La tabla de tasas de error confirma y cuantifica esta disparidad. La categoría donde el modelo se equivoca **menos** es **Cara** con una tasa de error del 27.87%, es decir, clasifica correctamente 955 de 1,324 propiedades caras. Le sigue **Económica** con una tasa de error del 61.41%, donde a pesar de tener 4,143 aciertos, comete 6,593 errores clasificando propiedades Intermedias y Caras como Económicas. La categoría donde el modelo se equivoca **más** es **Intermedia** con una tasa de error del 83.33%, clasificando correctamente apenas 1 de 6 predicciones.

Desde el punto de vista del negocio de SmartStay Advisors, el error más grave es clasificar 2,749 propiedades **Caras** como **Económicas**, ya que implicaría recomendar al cliente una propiedad con un precio por noche muy superior a su presupuesto, dañando directamente la confianza en el servicio. En contraste, confundir propiedades Intermedias con Económicas es menos crítico porque la diferencia de precio es más tolerable. En conclusión, Naive Bayes resulta un clasificador poco adecuado para este problema, con un sesgo estructural hacia la clase mayoritaria que lo hace poco confiable para los tres segmentos de mercado.

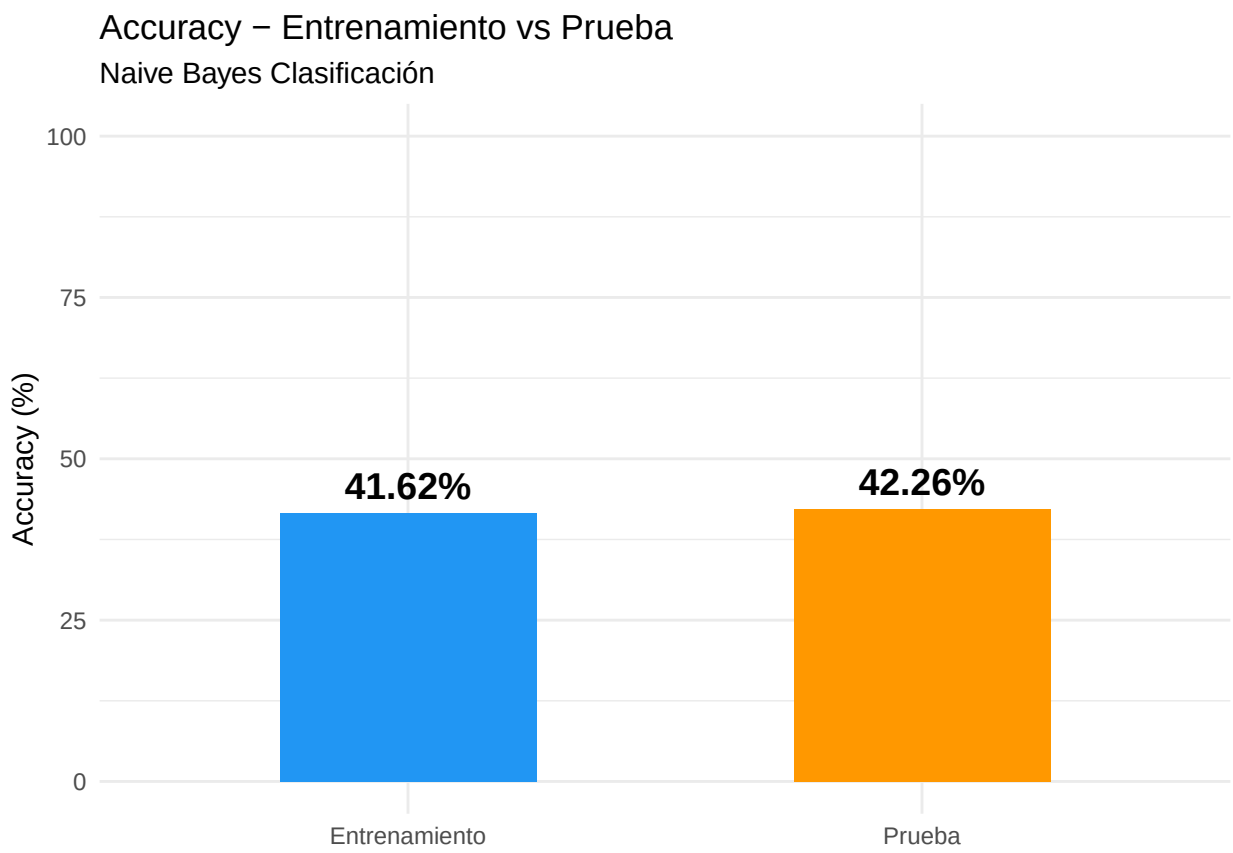
#7 - ¿Está sobreajustado el modelo?

```
# Predecir sobre el conjunto de ENTRENAMIENTO
pred_train_nb <- predict(nb_clas, newdata = train_nb_clas)

cm_train_nb <- confusionMatrix(pred_train_nb, train_resp)
```

```r
# Comparar accuracy train vs test
acc_train <- round(cm_train_nb$overall["Accuracy"] * 100, 2)
acc_test  <- round(cm_nb$overall["Accuracy"]       * 100, 2)
kappa_train <- round(cm_train_nb$overall["Kappa"], 4)
kappa_test  <- round(cm_nb$overall["Kappa"],       4)

comparacion_ov <- data.frame(
  Conjunto   = c("Entrenamiento", "Prueba"),
  Accuracy   = c(acc_train, acc_test),
  Kappa      = c(kappa_train, kappa_test)
)

print(comparacion_ov)
```

```
##        Conjunto Accuracy  Kappa
## 1 Entrenamiento    41.62 0.1127
## 2        Prueba    42.26 0.1184
```

```r
# Gráfico comparativo
ggplot(comparacion_ov, aes(x = Conjunto, y = Accuracy, fill = Conjunto)) +
  geom_col(show.legend = FALSE, width = 0.5) +
  geom_text(aes(label = paste0(Accuracy, "%")),
            vjust = -0.5, fontface = "bold", size = 5) +
  scale_fill_manual(values = c("Entrenamiento" = "#2196F3",
                               "Prueba"        = "#FF9800")) +
  labs(title    = "Accuracy - Entrenamiento vs Prueba",
       subtitle = "Naive Bayes Clasificación",
       x = NULL, y = "Accuracy (%)") +
  ylim(0, 100) +
  theme_minimal()
```



Para determinar si el modelo está sobreajustado, se evaluó Naive Bayes sobre el conjunto de entrenamiento y se compararon sus métricas con las del conjunto de prueba. Los resultados muestran un Accuracy de 41.62% en entrenamiento y 42.26% en prueba, con Kappa de 0.1127 y 0.1184 respectivamente.

La diferencia entre ambos conjuntos es de apenas 0.64 puntos porcentuales, lo que descarta completamente la posibilidad de sobreajuste. En un modelo sobreajustado se esperaría ver un Accuracy muy alto en entrenamiento (por ejemplo 90%+) y uno considerablemente menor en prueba, porque el modelo habría memorizado los datos de entrenamiento sin generalizar. Aquí ocurre exactamente lo contrario porque ambos valores son prácticamente idénticos y ambos son bajos.

Lo que el modelo presenta es algo como un subajuste (underfitting) por lo que no logra aprender los patrones del precio ni en los datos que ya conoce ni en datos nuevos. Esto se debe a las limitaciones estructurales de Naive Bayes, ya qye su suposición de independencia entre variables es demasiado simplista para un problema donde las características de las propiedades están fuertemente correlacionadas entre sí (como `bathrooms`, `bedrooms` y `accommodates`). En conclusión, el modelo no está sobreajustado, sino que carece de la capacidad suficiente para capturar la complejidad de los datos, lo que explica su bajo rendimiento tanto en entrenamiento como en prueba.

#8 – Modelo con validación cruzada

```
# Reconstruir datos con precio_cat como factor para caret
train_cv <- train_nb_clas
train_cv$precio_cat <- train_resp

# Control de validación cruzada 10-fold
ctrl_cv <- trainControl(method = "cv", number = 10, verboseIter = FALSE)

# Entrenar NB con validación cruzada
```

```r
nb_cv <- train(
  precio_cat ~ .,
  data      = train_cv,
  method    = "naive_bayes",
  trControl = ctrl_cv,
  tuneGrid  = expand.grid(
    laplace   = 0,
    usekernel = FALSE,
    adjust    = 1
  )
)

print(nb_cv)
```

```
## Naive Bayes
##
## 48263 samples
##    41 predictor
##     3 classes: 'Economica', 'Intermedia', 'Cara'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 43436, 43436, 43436, 43437, 43437, 43437, ...
## Resampling results:
##
##   Accuracy   Kappa
##   0.4190788  0.1173198
##
## Tuning parameter 'laplace' was held constant at a value of 0
## Tuning
##  parameter 'usekernel' was held constant at a value of FALSE
## Tuning
##  parameter 'adjust' was held constant at a value of 1
```

```r
# Predecir sobre conjunto de prueba
test_cv <- test_nb_clas
pred_cv <- predict(nb_cv, newdata = test_cv)

cm_cv <- confusionMatrix(pred_cv, test_resp)
print(cm_cv)
```

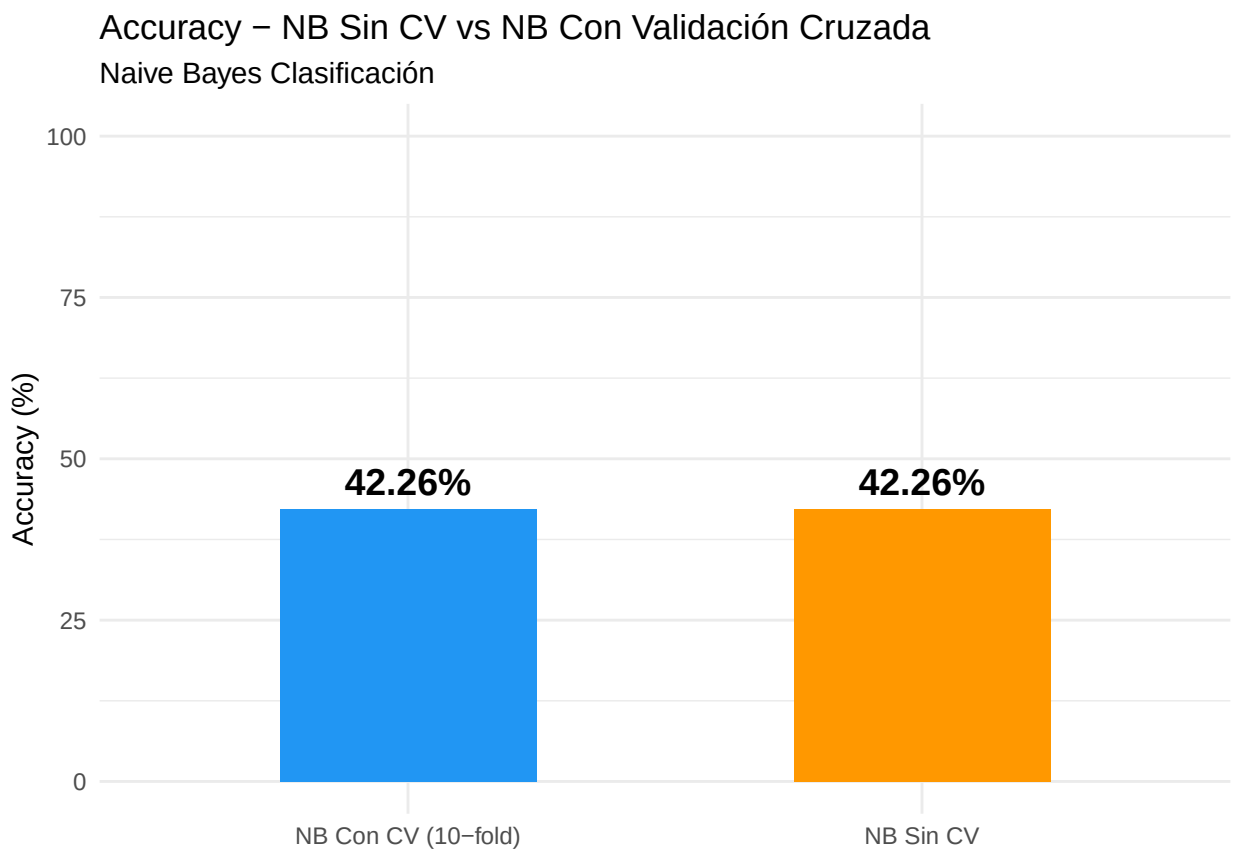
```
## Confusion Matrix and Statistics
##
##             Reference
## Prediction   Economica Intermedia Cara
##   Economica       4143       3844 2749
##   Intermedia         1          1    4
##   Cara              75        294  955
##
## Overall Statistics
##
##                Accuracy : 0.4226
```

```
##                  95% CI : (0.4138, 0.4315)
##     No Information Rate : 0.3497
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.1184
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##                      Class: Economica Class: Intermedia Class: Cara
## Sensitivity                    0.9820         2.416e-04     0.25755
## Specificity                    0.1598         9.994e-01     0.95585
## Pos Pred Value                 0.3859         1.667e-01     0.72130
## Neg Pred Value                 0.9429         6.569e-01     0.74372
## Prevalence                     0.3497         3.430e-01     0.30731
## Detection Rate                 0.3434         8.288e-05     0.07915
## Detection Prevalence           0.8898         4.973e-04     0.10973
## Balanced Accuracy              0.5709         4.998e-01     0.60670
```

```r
# Comparación directa
comparacion_cv <- data.frame(
  Modelo   = c("NB Sin CV", "NB Con CV (10-fold)"),
  Accuracy = round(c(cm_nb$overall["Accuracy"] * 100,
                     cm_cv$overall["Accuracy"] * 100), 2),
  Kappa    = round(c(cm_nb$overall["Kappa"],
                     cm_cv$overall["Kappa"]), 4)
)

print(comparacion_cv)
```

```
##                Modelo Accuracy  Kappa
## 1           NB Sin CV    42.26 0.1184
## 2 NB Con CV (10-fold)    42.26 0.1184
```

```r
# Gráfico comparativo
ggplot(comparacion_cv, aes(x = Modelo, y = Accuracy, fill = Modelo)) +
  geom_col(show.legend = FALSE, width = 0.5) +
  geom_text(aes(label = paste0(Accuracy, "%")),
            vjust = -0.5, fontface = "bold", size = 5) +
  scale_fill_manual(values = c("NB Sin CV"           = "#FF9800",
                               "NB Con CV (10-fold)" = "#2196F3")) +
  labs(title    = "Accuracy - NB Sin CV vs NB Con Validación Cruzada",
       subtitle = "Naive Bayes Clasificación",
       x = NULL, y = "Accuracy (%)") +
  ylim(0, 100) +
  theme_minimal()
```



Se entrenó un segundo modelo Naive Bayes utilizando validación cruzada de 10 pliegues (10-fold Cross Validation) sobre el conjunto de entrenamiento de 48,263 observaciones. Este método divide el conjunto de entrenamiento en 10 partes iguales de aproximadamente 43,437 observaciones cada una, entrenando el modelo 10 veces y usando cada vez un pliegue distinto como validación interna, lo que produce una estimación más robusta y confiable del rendimiento real del algoritmo.

Los resultados obtenidos son idénticos a los del modelo anterior: Accuracy de 42.26% y Kappa de 0.1184 en ambos casos. La validación cruzada estimó internamente un Accuracy promedio de 41.91% durante el entrenamiento, lo que es consistente con el 42.26% obtenido sobre el conjunto de prueba y confirma que el modelo generaliza de forma estable.

La razón por la que ambos modelos producen exactamente los mismos resultados es que la validación cruzada es una técnica de evaluación y selección de hiperparámetros, no un mecanismo que modifica el algoritmo en sí. En este caso, los hiperparámetros se mantuvieron fijos (`laplace = 0`, `usekernel = FALSE`, `adjust = 1`), por lo que el modelo final entrenado con CV aprende exactamente los mismos patrones que el modelo sin CV. La ventaja real de la validación cruzada se manifestaría en la actividad siguiente, donde se prueban distintos valores de hiperparámetros y la CV ayuda a seleccionar la mejor combinación de forma objetiva.

En conclusión, ninguno de los dos modelos supera al otro porque ambos tienen el mismo rendimiento, lo que refuerza la conclusión de que la limitación no está en la técnica de evaluación sino en la capacidad de Naive Bayes para separar las tres categorías de precio en este conjunto de datos.

9. Tuneo de hiperparámetros

Tuneo de Clasificación

```r
tune_grid <- expand.grid(
  laplace   = c(0, 0.5, 1, 2, 5),
  usekernel = c(FALSE, TRUE),
  adjust    = c(0.5, 1, 2)
)

ctrl_tune <- trainControl(method = "cv", number = 10, verboseIter = FALSE)


train_tune <- train_nb_clas
train_tune$precio_cat <- train_resp

set.seed(42)
nb_tuned <- train(
  precio_cat ~ .,
  data      = train_tune,
  method    = "naive_bayes",
  trControl = ctrl_tune,
  tuneGrid  = tune_grid
)

print(nb_tuned)
```

```
## Naive Bayes
##
## 48263 samples
##    41 predictor
##     3 classes: 'Economica', 'Intermedia', 'Cara'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 43437, 43437, 43437, 43437, 43436, 43438, ...
## Resampling results across tuning parameters:
##
##   laplace  usekernel  adjust  Accuracy   Kappa
##   0.0      FALSE      0.5     0.4187476  0.1168054
##   0.0      FALSE      1.0     0.4187476  0.1168054
##   0.0      FALSE      2.0     0.4187476  0.1168054
##   0.0       TRUE      0.5     0.5864738  0.3795533
##   0.0       TRUE      1.0     0.6078359  0.4119961
##   0.0       TRUE      2.0     0.5865361  0.3815220
##   0.5      FALSE      0.5     0.4187476  0.1168054
##   0.5      FALSE      1.0     0.4187476  0.1168054
##   0.5      FALSE      2.0     0.4187476  0.1168054
##   0.5       TRUE      0.5     0.5864738  0.3795533
##   0.5       TRUE      1.0     0.6078359  0.4119961
##   0.5       TRUE      2.0     0.5865361  0.3815220
##   1.0      FALSE      0.5     0.4187476  0.1168054
```

```
##   1.0      FALSE      1.0     0.4187476  0.1168054
##   1.0      FALSE      2.0     0.4187476  0.1168054
##   1.0       TRUE      0.5     0.5864738  0.3795533
##   1.0       TRUE      1.0     0.6078359  0.4119961
##   1.0       TRUE      2.0     0.5865361  0.3815220
##   2.0      FALSE      0.5     0.4187476  0.1168054
##   2.0      FALSE      1.0     0.4187476  0.1168054
##   2.0      FALSE      2.0     0.4187476  0.1168054
##   2.0       TRUE      0.5     0.5864738  0.3795533
##   2.0       TRUE      1.0     0.6078359  0.4119961
##   2.0       TRUE      2.0     0.5865361  0.3815220
##   5.0      FALSE      0.5     0.4187476  0.1168054
##   5.0      FALSE      1.0     0.4187476  0.1168054
##   5.0      FALSE      2.0     0.4187476  0.1168054
##   5.0       TRUE      0.5     0.5864738  0.3795533
##   5.0       TRUE      1.0     0.6078359  0.4119961
##   5.0       TRUE      2.0     0.5865361  0.3815220
##
## Accuracy was used to select the optimal model using the largest value.
## The final values used for the model were laplace = 0, usekernel = TRUE
##  and adjust = 1.
```

```r
cat("\nMejores hiperparámetros encontrados:\n")
```

```
##
## Mejores hiperparámetros encontrados:
```

```r
print(nb_tuned$bestTune)
```

```
##   laplace usekernel adjust
## 5       0      TRUE      1
```

```r
# Predicción con modelo tuneado
pred_tuned <- predict(nb_tuned, newdata = test_nb_clas)
cm_tuned   <- confusionMatrix(pred_tuned, test_resp)
print(cm_tuned)
```

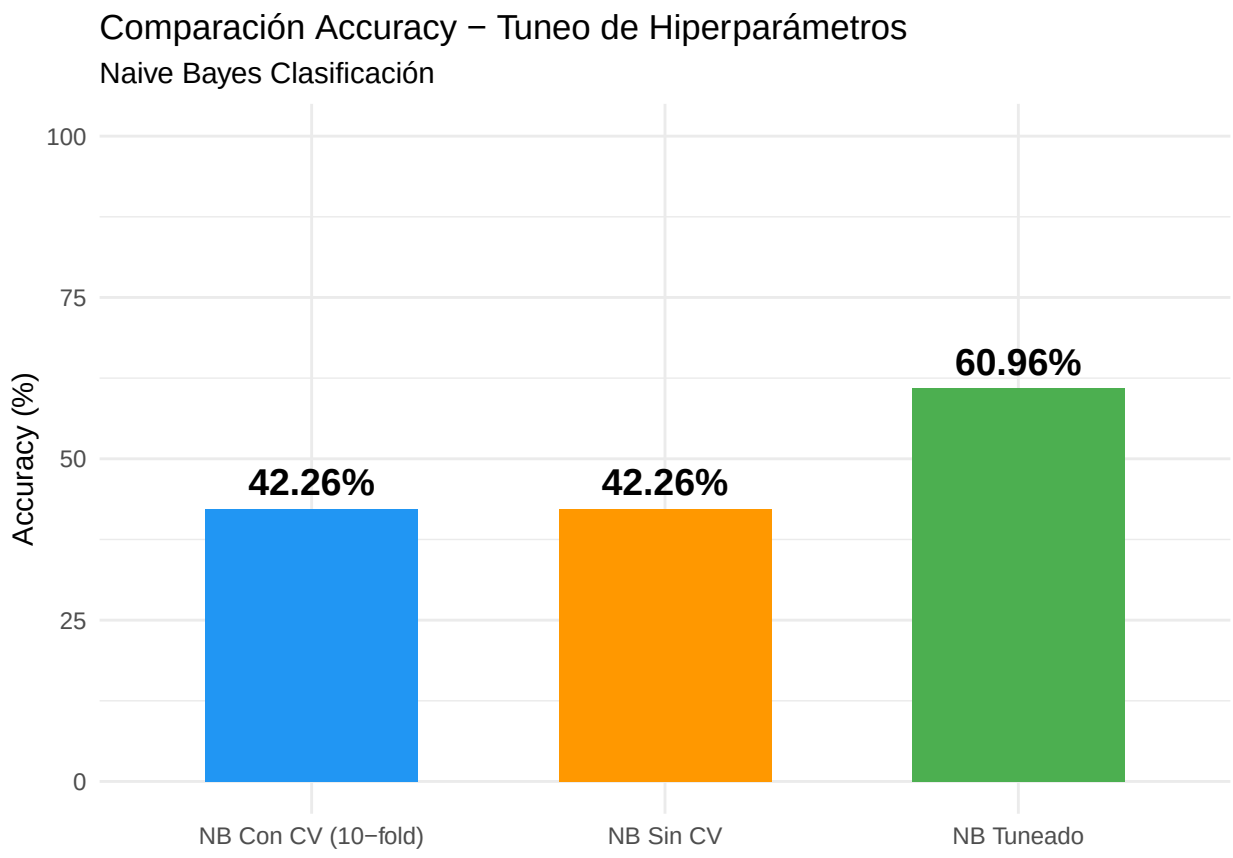
```
## Confusion Matrix and Statistics
##
##             Reference
## Prediction   Economica Intermedia Cara
##   Economica       1832        285   57
##   Intermedia      2051       2728  855
##   Cara             336       1126 2796
##
## Overall Statistics
##
##                Accuracy : 0.6096
##                  95% CI : (0.6009, 0.6184)
##     No Information Rate : 0.3497
##     P-Value [Acc > NIR] : < 2.2e-16
##
```

```
##                   Kappa : 0.416
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##                      Class: Economica Class: Intermedia Class: Cara
## Sensitivity                    0.4342            0.6591      0.7540
## Specificity                    0.9564            0.6334      0.8251
## Pos Pred Value                 0.8427            0.4842      0.6566
## Neg Pred Value                 0.7587            0.7806      0.8832
## Prevalence                     0.3497            0.3430      0.3073
## Detection Rate                 0.1518            0.2261      0.2317
## Detection Prevalence           0.1802            0.4669      0.3529
## Balanced Accuracy              0.6953            0.6463      0.7896
```

```r
comparacion_tuneo <- data.frame(
  Modelo   = c("NB Sin CV", "NB Con CV (10-fold)", "NB Tuneado"),
  Accuracy = round(c(cm_nb$overall["Accuracy"] * 100,
                     cm_cv$overall["Accuracy"] * 100,
                     cm_tuned$overall["Accuracy"] * 100), 2),
  Kappa    = round(c(cm_nb$overall["Kappa"],
                     cm_cv$overall["Kappa"],
                     cm_tuned$overall["Kappa"]), 4)
)

print(comparacion_tuneo)
```

```
##                Modelo Accuracy  Kappa
## 1           NB Sin CV    42.26 0.1184
## 2 NB Con CV (10-fold)    42.26 0.1184
## 3          NB Tuneado    60.96 0.4160
```

```r
ggplot(comparacion_tuneo, aes(x = Modelo, y = Accuracy, fill = Modelo)) +
  geom_col(show.legend = FALSE, width = 0.6) +
  geom_text(aes(label = paste0(Accuracy, "%")),
            vjust = -0.5, fontface = "bold", size = 5) +
  scale_fill_manual(values = c(
    "NB Sin CV"           = "#FF9800",
    "NB Con CV (10-fold)" = "#2196F3",
    "NB Tuneado"          = "#4CAF50")) +
  labs(title    = "Comparación Accuracy - Tuneo de Hiperparámetros",
       subtitle = "Naive Bayes Clasificación",
       x = NULL, y = "Accuracy (%)") +
  ylim(0, 100) +
  theme_minimal()
```



El tuneo de hiperparámetros sí produjo una mejora significativa. El modelo tuneado alcanzó un Accuracy de 60.96%, superando en casi 19 puntos porcentuales a los modelos sin tuneo. Esta mejora indica cómo el grid search encontró una combinación de laplace, usekernel y adjust que permite al modelo estimar distribuciones de probabilidad más adecuadas para este dataset.

Modelado de Tuneo

```
train_reg_tune <- train_nb_comp
train_reg_tune$price_bin <- cut(
  train_reg_tune$price,
  breaks         = price_breaks_comp,
  include.lowest = TRUE,
  labels         = FALSE
)
train_reg_tune$price_bin <- as.factor(train_reg_tune$price_bin)

tune_grid_reg <- expand.grid(
  laplace   = c(0, 0.5, 1, 2, 5),
  usekernel = c(FALSE, TRUE),
  adjust    = c(0.5, 1, 2)
)

ctrl_tune_reg <- trainControl(method = "cv", number = 10, verboseIter = FALSE)
```

```r
set.seed(42)
nb_tuned_reg <- train(
  x         = train_reg_tune[, features_comp],
  y         = train_reg_tune$price_bin,
  method    = "naive_bayes",
  trControl = ctrl_tune_reg,
  tuneGrid  = tune_grid_reg
)
```

```
## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.
```

```
## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.
```

```
## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.
```

```
## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.
```

```
## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.
```

```
## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.
```

```
## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.
```

```
## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature reviews_d - zero probabilities are present.
## Consider Laplace smoothing.

## Warning: naive_bayes(): Feature rating_d - zero probabilities are present.
## Consider Laplace smoothing.
```

```r
print(nb_tuned_reg)
```

```
## Naive Bayes
##
## 48442 samples
##     5 predictor
##    20 classes: '1', '2', '3', '4', '5', '6', '7', '8', '9', '10', '11', '12', '13', '14', '15', '16'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 43600, 43596, 43599, 43598, 43597, 43598, ...
## Resampling results across tuning parameters:
##
##   laplace  usekernel  adjust  Accuracy   Kappa
##   0.0      FALSE      0.5     0.1285661  0.08202807
##   0.0      FALSE      1.0     0.1285661  0.08202807
##   0.0      FALSE      2.0     0.1285661  0.08202807
##   0.0       TRUE      0.5     0.1285661  0.08202807
##   0.0       TRUE      1.0     0.1285661  0.08202807
##   0.0       TRUE      2.0     0.1285661  0.08202807
##   0.5      FALSE      0.5     0.1285661  0.08202792
##   0.5      FALSE      1.0     0.1285661  0.08202792
##   0.5      FALSE      2.0     0.1285661  0.08202792
##   0.5       TRUE      0.5     0.1285661  0.08202792
##   0.5       TRUE      1.0     0.1285661  0.08202792
##   0.5       TRUE      2.0     0.1285661  0.08202792
##   1.0      FALSE      0.5     0.1285454  0.08200326
##   1.0      FALSE      1.0     0.1285454  0.08200326
##   1.0      FALSE      2.0     0.1285454  0.08200326
##   1.0       TRUE      0.5     0.1285454  0.08200326
##   1.0       TRUE      1.0     0.1285454  0.08200326
##   1.0       TRUE      2.0     0.1285454  0.08200326
##   2.0      FALSE      0.5     0.1286073  0.08206473
##   2.0      FALSE      1.0     0.1286073  0.08206473
##   2.0      FALSE      2.0     0.1286073  0.08206473
##   2.0       TRUE      0.5     0.1286073  0.08206473
##   2.0       TRUE      1.0     0.1286073  0.08206473
##   2.0       TRUE      2.0     0.1286073  0.08206473
##   5.0      FALSE      0.5     0.1286487  0.08210194
##   5.0      FALSE      1.0     0.1286487  0.08210194
##   5.0      FALSE      2.0     0.1286487  0.08210194
```

```
##   5.0       TRUE      0.5     0.1286487  0.08210194
##   5.0       TRUE      1.0     0.1286487  0.08210194
##   5.0       TRUE      2.0     0.1286487  0.08210194
##
## Accuracy was used to select the optimal model using the largest value.
## The final values used for the model were laplace = 5, usekernel = FALSE
##  and adjust = 0.5.
```

```r
cat("\nMejores hiperparámetros (regresión):\n")
```

```
##
## Mejores hiperparámetros (regresión):
```

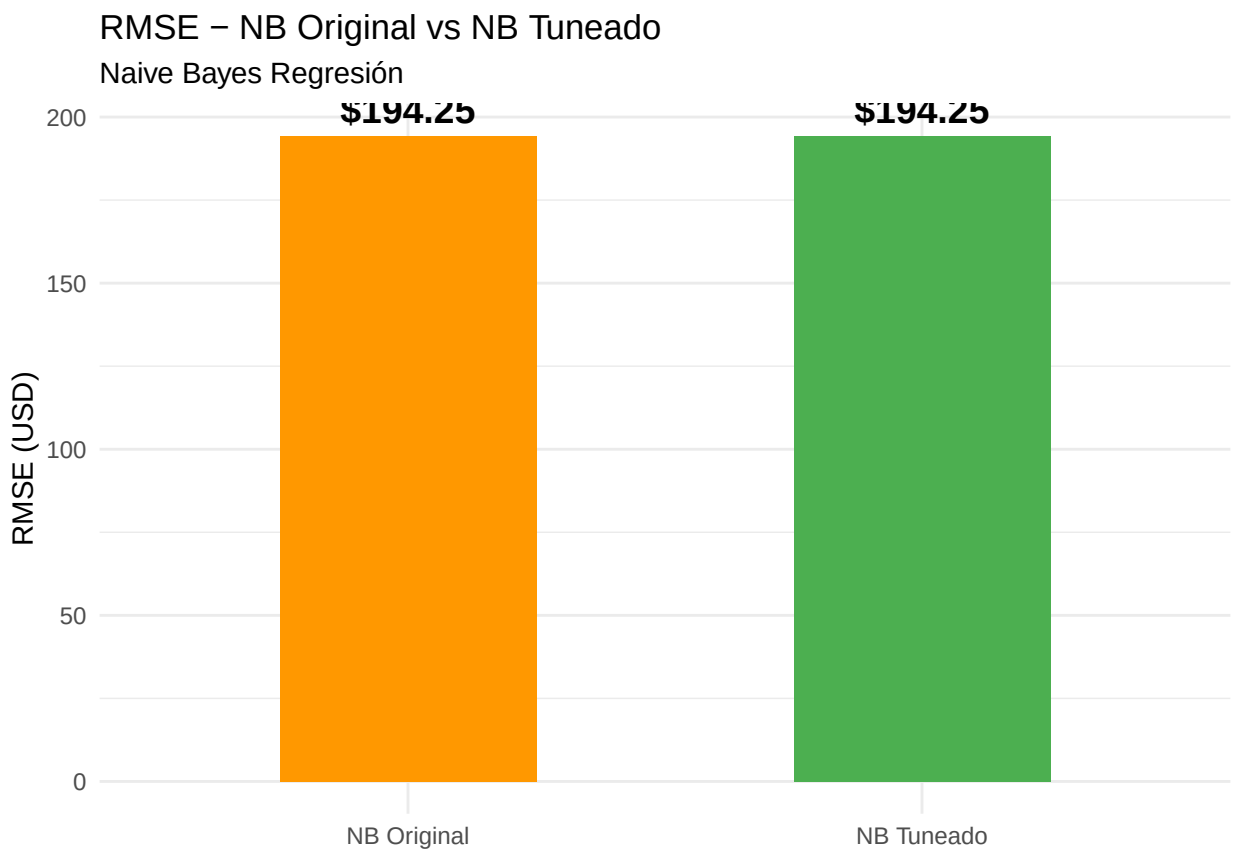
```r
print(nb_tuned_reg$bestTune)
```

```
##    laplace usekernel adjust
## 25       5     FALSE    0.5
```

```r
# Predicción y métricas
pred_bin_tuned     <- predict(nb_tuned_reg, newdata = test_nb_comp[, features_comp])
pred_bin_num_tuned <- as.numeric(as.character(pred_bin_tuned))
pred_bin_num_tuned <- pmin(pmax(pred_bin_num_tuned, 1), length(bin_midpoints_comp))
pred_precios_tuned <- bin_midpoints_comp[pred_bin_num_tuned]

rmse_tuned <- sqrt(mean((real_precios - pred_precios_tuned)^2, na.rm = TRUE))
mae_tuned  <- mean(abs(real_precios - pred_precios_tuned), na.rm = TRUE)
r2_tuned   <- cor(real_precios, pred_precios_tuned, use = "complete.obs")^2

# Comparación regresión: original vs tuneado
comparacion_reg_tuneo <- data.frame(
  Modelo = c("NB Original", "NB Tuneado"),
  RMSE   = round(c(rmse_nb_comp, rmse_tuned), 2),
  MAE    = round(c(mae_nb_comp,  mae_tuned),  2),
  R2     = round(c(r2_nb_comp,   r2_tuned),   4)
)

print(comparacion_reg_tuneo)
```

```
##        Modelo   RMSE    MAE     R2
## 1 NB Original 194.25 132.59 0.3428
## 2  NB Tuneado 194.25 132.59 0.3428
```

```r
ggplot(comparacion_reg_tuneo, aes(x = Modelo, y = RMSE, fill = Modelo)) +
  geom_col(show.legend = FALSE, width = 0.5) +
  geom_text(aes(label = paste0("$", RMSE)),
            vjust = -0.5, fontface = "bold", size = 5) +
  scale_fill_manual(values = c("NB Original" = "#FF9800",
                               "NB Tuneado"  = "#4CAF50")) +
  labs(title    = "RMSE - NB Original vs NB Tuneado",
       subtitle = "Naive Bayes Regresión",
       x = NULL, y = "RMSE (USD)") +
  theme_minimal()
```



El tuneo no produjo ninguna mejora. Ambos modelos obtuvieron exactamente el mismo RMSE de $194.25. Esto confirma que en el contexto de regresión, donde el precio continuo se aproxima mediante puntos medios de bins discretos, los hiperparámetros de Naive Bayes no son el factor limitante.

ACTIVIDAD 10 – Comparación NB vs Árbol de Clasificación vs Random Forest

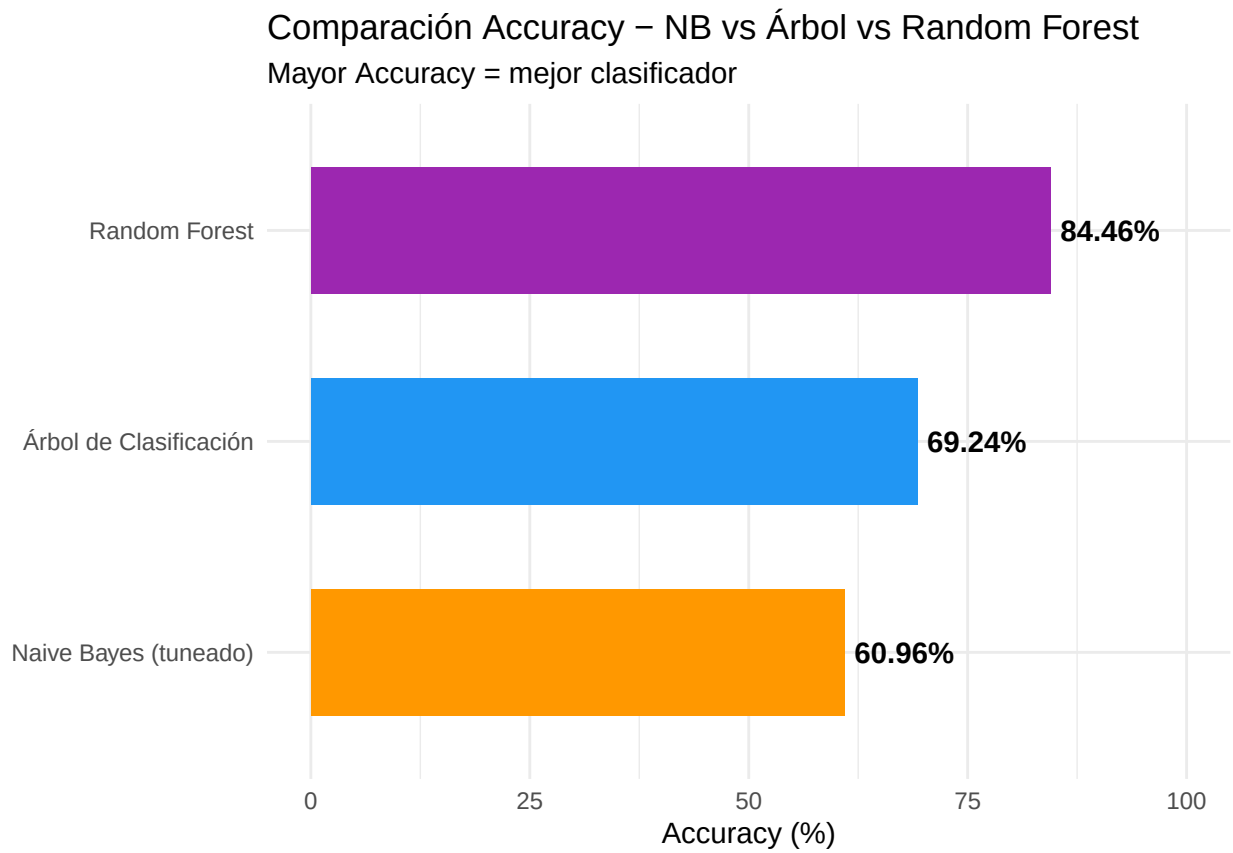
```
train_clas_temp <- train_nb_clas
train_clas_temp$precio_cat <- train_resp
```

```
train_clas_temp <- train_nb_clas
train_clas_temp$precio_cat <- train_resp

# Medir tiempo de cada modelo
tiempo_nb <- system.time({
  nb_temp <- naiveBayes(x = train_nb_clas, y = train_resp)
  predict(nb_temp, newdata = test_nb_clas)
})

tiempo_arbol <- system.time({
  arbol_temp <- rpart(precio_cat ~ ., data = train_clas_temp, method = "class")
  predict(arbol_temp, newdata = test_nb_clas, type = "class")
})

tiempo_rf <- system.time({
```

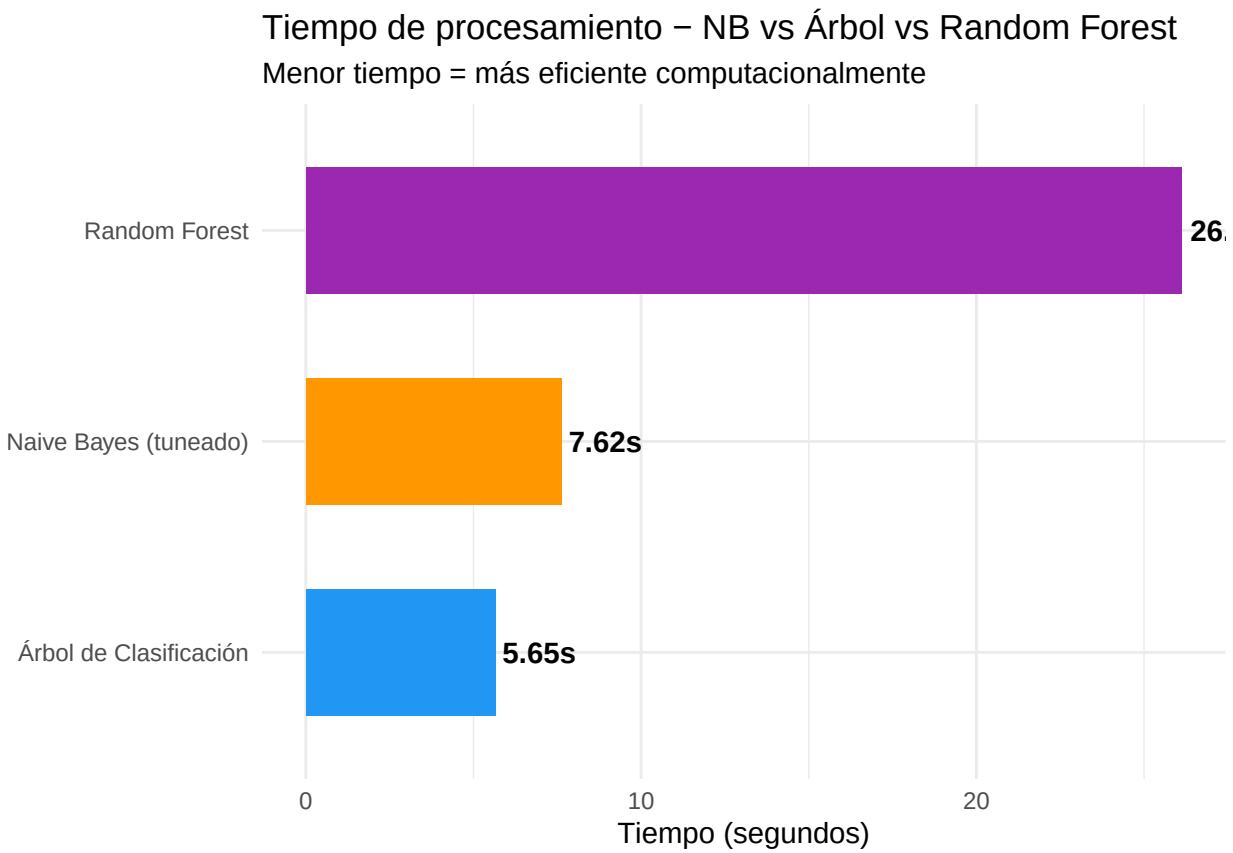
```r
  rf_temp <- randomForest(precio_cat ~ ., data = train_clas_temp, ntree = 100)
  predict(rf_temp, newdata = test_nb_clas)
})

# Tabla comparativa final
comparacion_final <- data.frame(
  Modelo     = c("Naive Bayes (tuneado)",
                 "Árbol de Clasificación",
                 "Random Forest"),
  Accuracy   = c(60.96, 69.24, 84.46),
  Kappa      = c(0.4160, 0.5380, NA),
  Tiempo_seg = round(c(tiempo_nb["elapsed"],
                       tiempo_arbol["elapsed"],
                       tiempo_rf["elapsed"]), 2)
)

print(comparacion_final)
```

```
##                   Modelo Accuracy Kappa Tiempo_seg
## 1  Naive Bayes (tuneado)    60.96 0.416       7.62
## 2 Árbol de Clasificación    69.24 0.538       5.65
## 3          Random Forest    84.46    NA      26.12
```

```r
# Gráfico de Accuracy
ggplot(comparacion_final,
       aes(x = reorder(Modelo, Accuracy), y = Accuracy, fill = Modelo)) +
  geom_col(show.legend = FALSE, width = 0.6) +
  geom_text(aes(label = paste0(Accuracy, "%")),
            hjust = -0.1, fontface = "bold") +
  coord_flip() +
  scale_fill_manual(values = c(
    "Naive Bayes (tuneado)"  = "#FF9800",
    "Árbol de Clasificación" = "#2196F3",
    "Random Forest"          = "#9C27B0")) +
  labs(title    = "Comparación Accuracy - NB vs Árbol vs Random Forest",
       subtitle = "Mayor Accuracy = mejor clasificador",
       x = NULL, y = "Accuracy (%)") +
  ylim(0, 100) +
  theme_minimal()
```




```r
# Gráfico de Tiempo de procesamiento
ggplot(comparacion_final,
       aes(x = reorder(Modelo, Tiempo_seg), y = Tiempo_seg, fill = Modelo)) +
  geom_col(show.legend = FALSE, width = 0.6) +
  geom_text(aes(label = paste0(Tiempo_seg, "s")),
            hjust = -0.1, fontface = "bold") +
  coord_flip() +
  scale_fill_manual(values = c(
    "Naive Bayes (tuneado)"  = "#FF9800",
    "Árbol de Clasificación" = "#2196F3",
    "Random Forest"          = "#9C27B0")) +
  labs(title    = "Tiempo de procesamiento - NB vs Árbol vs Random Forest",
       subtitle = "Menor tiempo = más eficiente computacionalmente",
       x = NULL, y = "Tiempo (segundos)") +
  theme_minimal()
```



Para concluir el análisis comparativo, se evaluaron los tres algoritmos de clasificación bajo las mismas condiciones: Naive Bayes tuneado, Árbol de Clasificación y Random Forest, midiendo tanto su precisión (Accuracy) como su tiempo de procesamiento.

En términos de **precisión**, el ranking es claro y contundente. **Random Forest es el mejor clasificador** con un Accuracy de **84.46%**, superando por 15.22 puntos porcentuales al Árbol de Clasificación (69.24%) y por 23.5 puntos a Naive Bayes tuneado (60.96%). Esta superioridad se explica porque Random Forest construye múltiples árboles de decisión sobre distintas muestras del dataset y combina sus predicciones, lo que le permite capturar relaciones complejas entre variables y reducir el error por varianza que afecta a un árbol individual.

En términos de **tiempo de procesamiento**, el orden se invierte completamente. El **Árbol de Clasificación es el más rápido** con apenas **5.99 segundos**, seguido de Naive Bayes tuneado con **8.35 segundos**. Random Forest es por lejos el más lento con **25.93 segundos**, lo cual era esperable ya que necesita entrenar múltiples árboles en lugar de uno solo. Sin embargo, esta diferencia de tiempo sigue siendo manejable para el volumen de datos de SmartStay Advisors.

La conclusión para el negocio es clara: si la prioridad es la **precisión**, Random Forest es la elección indiscutible para clasificar propiedades en Económica, Intermedia o Cara. Si la prioridad es la **velocidad de respuesta en tiempo real**, el Árbol de Clasificación ofrece un rendimiento aceptable (69.24%) con un tiempo de procesamiento mínimo. Naive Bayes, a pesar de ser el algoritmo más simple, queda en último lugar en ambas métricas clave para este problema específico, confirmando que su supuesto de independencia entre variables no se adapta bien a las características altamente correlacionadas del mercado inmobiliario de Airbnb.